

Data Mining 2026 — Project Notebook

Course: Data Mining

Project track: ☒ Standard Analysis ☐ Research-Oriented

Group Number: 16 **Group members:**

- Giancarlo Marchesini
- Himan Kalita
- Kris Kraack

Task Description

Objective (Module 2 perspective) Using the Bike Sharing Dataset (2011-2012), this module models daily records as a graph to analyse demand structure from a graph perspective. Each node represents one day, and edges connect days with similar weather and calendar context conditions. The objective is to identify recurring day types and explain their behavioral meaning using post-hoc demand attributes.

Research question Can a day-level similarity graph reveal a small and meaningful set of recurring day types that differ in total demand, rider composition, and weather/calendar context?

Hypotheses

- **H1 (structural):** Days with similar weather and calendar conditions form coherent graph communities.
- **H2 (behavioral):** The detected communities show clear differences in total bike rentals (cnt) and rider composition between casual riders (casual) and registered riders (registered).
- **H3 (interpretability):** The graph communities can be interpreted as practical demand regimes (e.g., commuter-dominant vs leisure-dominant day types), rather than arbitrary partitions.

0. Reproducibility and Setup

```
import sys
print(sys.version)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
```

```
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import silhouette_score
from sklearn.neighbors import NearestNeighbors
from sklearn.manifold import TSNE
from sklearn.cluster import DBSCAN
from sklearn.cluster import SpectralClustering #####
```

```
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```

```
3.11.7 (main, Dec 15 2023, 12:09:56) [Clang 14.0.6 ]
```

1. Dataset Description and Loading

1.1 Dataset Overview

- Dataset: Bike Sharing Dataset (2011–2012)
- Reference: Fanaee-T, H. (2013). Bike Sharing [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5W894>.
- Source: <https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset>
- Number of instances:
 - 'hour.csv' : 17379;
 - 'day.csv' : 731.
- Number of features / entities:
 - 'hour.csv' : 17;
 - 'day.csv' : 16.
- Missing values: The dataset doesn't have missing values.
- Basic statistics: See section below.

Dataset characteristics Both hour.csv and day.csv have the following fields, except hr which is not available in day.csv

- instant: record index
- dteday : date
- season : season (1:springer, 2:summer, 3:fall, 4:winter)
- yr : year (0: 2011, 1:2012)
- mnth : month (1 to 12)
- hr : hour (0 to 23)
- holiday : weather day is holiday or not (extracted from <http://dchr.dc.gov/page/holiday-schedule>)
- weekday : day of the week
- workingday : if day is neither weekend nor holiday is 1, otherwise is 0.
- weathersit :
 - 1: Clear, Few clouds, Partly cloudy, Partly cloudy
 - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist

- 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds
- 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog
- temp : Normalized temperature in Celsius. The values are divided to 41 (max)
- atemp: Normalized feeling temperature in Celsius. The values are divided to 50 (max)
- hum: Normalized humidity. The values are divided to 100 (max)
- windspeed: Normalized wind speed. The values are divided to 67 (max)
- casual: count of casual users
- registered: count of registered users
- cnt: count of total rental bikes including both casual and registered

```
# Load dataset
data_day = pd.read_csv('./day.csv') # day csv file
data_hour = pd.read_csv('./hour.csv') # hour csv file

# Print dataset(s) shapes (r,c)
print(f"day dataset - shape: {data_day.shape}") #day
print(f"hour dataset - shape: {data_hour.shape}") #hour

# Print missing values (NaNs)
print("day dataset - NaNs:", data_day.isnull().sum().sum())
print("hour dataset - NaNs:", data_hour.isnull().sum().sum())

# Inspect
#data_hour.head(5)
data_day.head(5)

day dataset - shape: (731, 16)
hour dataset - shape: (17379, 17)
day dataset - NaNs: 0
hour dataset - NaNs: 0
```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday
0	1	2011-01-01	1	0	1	0	6	0
1	2	2011-01-02	1	0	1	0	0	0
2	3	2011-01-03	1	0	1	0	1	1
3	4	2011-01-04	1	0	1	0	2	1
4	5	2011-01-05	1	0	1	0	3	1

	weathersit	temp	atemp	hum	windspeed	casual
0	2	0.344167	0.363625	0.805833	0.160446	331
654						
1	2	0.363478	0.353739	0.696087	0.248539	131

```

670
2      1  0.196364  0.189405  0.437273  0.248309  120
1229
3      1  0.200000  0.212122  0.590435  0.160296  108
1454
4      1  0.226957  0.229270  0.436957  0.186900  82
1518

    cnt
0    985
1    801
2   1349
3   1562
4   1600

```

Exploratory Data Analysis

Basic statistics

```

# Basic statistics: Day
day_stats = (
    data_day.select_dtypes(include='number')
            .describe()
            .T[['count', 'mean', 'std', 'min', '25%', '50%', '75%',
'max']]
            .round(3)
)
print("Basic statistics: Day:")
display(day_stats)

# Basic statistics: hour
hour_stats = (
    data_hour.select_dtypes(include='number')
            .describe()
            .T[['count', 'mean', 'std', 'min', '25%', '50%', '75%',
'max']]
            .round(3)
)
print("Basic statistics: Hour")
display(hour_stats)

```

Basic statistics: Day:

	count	mean	std	min	25%	50%
75% \ instant	731.0	366.000	211.166	1.000	183.500	366.000
548.500 season	731.0	2.497	1.111	1.000	2.000	3.000
3.000 yr	731.0	0.501	0.500	0.000	0.000	1.000

1.000						
mnth	731.0	6.520	3.452	1.000	4.000	7.000
10.000						
holiday	731.0	0.029	0.167	0.000	0.000	0.000
0.000						
weekday	731.0	2.997	2.005	0.000	1.000	3.000
5.000						
workingday	731.0	0.684	0.465	0.000	0.000	1.000
1.000						
weathersit	731.0	1.395	0.545	1.000	1.000	1.000
2.000						
temp	731.0	0.495	0.183	0.059	0.337	0.498
0.655						
atemp	731.0	0.474	0.163	0.079	0.338	0.487
0.609						
hum	731.0	0.628	0.142	0.000	0.520	0.627
0.730						
windspeed	731.0	0.190	0.077	0.022	0.135	0.181
0.233						
casual	731.0	848.176	686.622	2.000	315.500	713.000
1096.000						
registered	731.0	3656.172	1560.256	20.000	2497.000	3662.000
4776.500						
cnt	731.0	4504.349	1937.211	22.000	3152.000	4548.000
5956.000						

	max
instant	731.000
season	4.000
yr	1.000
mnth	12.000
holiday	1.000
weekday	6.000
workingday	1.000
weathersit	3.000
temp	0.862
atemp	0.841
hum	0.972
windspeed	0.507
casual	3410.000
registered	6946.000
cnt	8714.000

Basic statistics: Hour

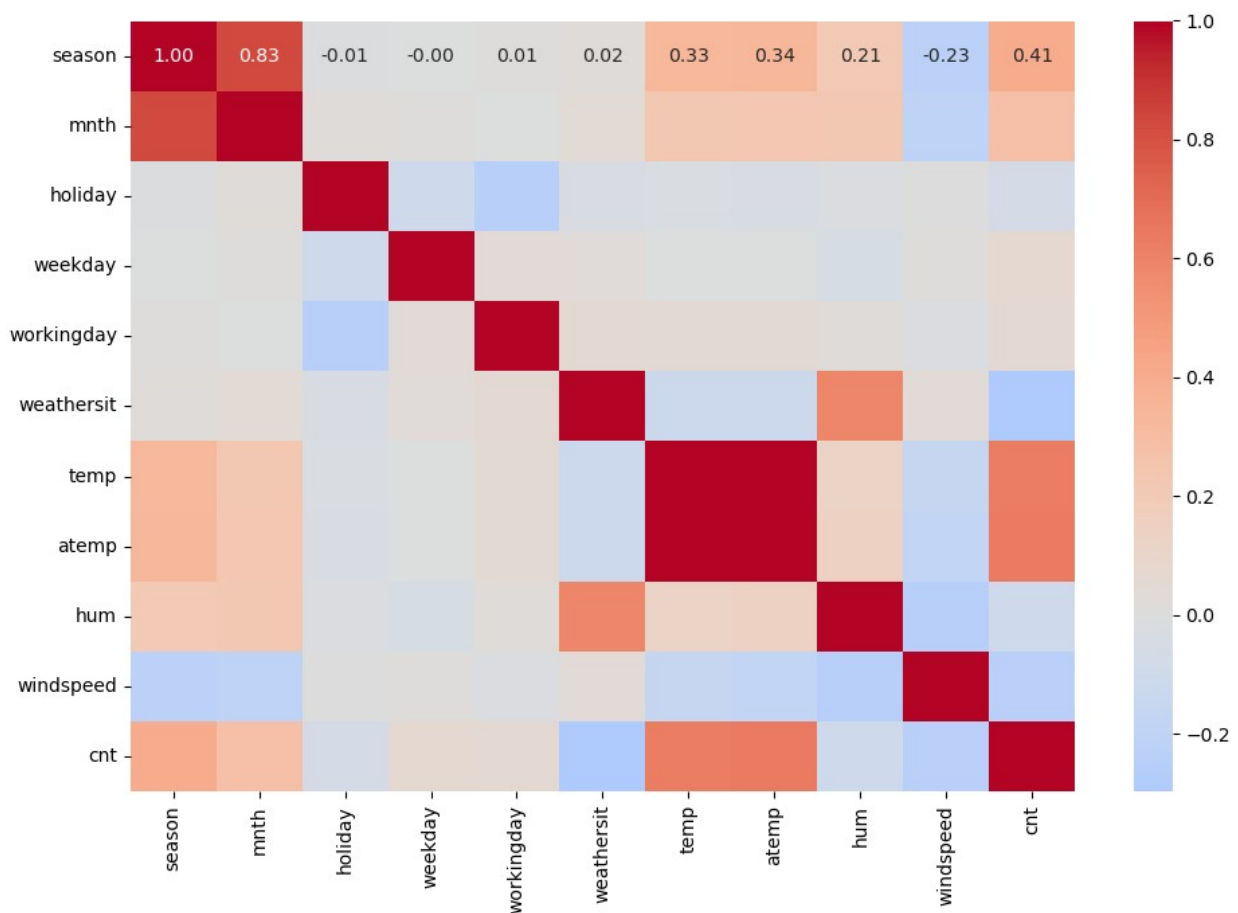
	count	mean	std	min	25%	50%
75% \						
instant	17379.0	8690.000	5017.029	1.00	4345.500	8690.000
13034.500						
season	17379.0	2.502	1.107	1.00	2.000	3.000

3.000						
yr	17379.0	0.503	0.500	0.00	0.000	1.000
1.000						
mnth	17379.0	6.538	3.439	1.00	4.000	7.000
10.000						
hr	17379.0	11.547	6.914	0.00	6.000	12.000
18.000						
holiday	17379.0	0.029	0.167	0.00	0.000	0.000
0.000						
weekday	17379.0	3.004	2.006	0.00	1.000	3.000
5.000						
workingday	17379.0	0.683	0.465	0.00	0.000	1.000
1.000						
weathersit	17379.0	1.425	0.639	1.00	1.000	1.000
2.000						
temp	17379.0	0.497	0.193	0.02	0.340	0.500
0.660						
atemp	17379.0	0.476	0.172	0.00	0.333	0.485
0.621						
hum	17379.0	0.627	0.193	0.00	0.480	0.630
0.780						
windspeed	17379.0	0.190	0.122	0.00	0.104	0.194
0.254						
casual	17379.0	35.676	49.305	0.00	4.000	17.000
48.000						
registered	17379.0	153.787	151.357	0.00	34.000	115.000
220.000						
cnt	17379.0	189.463	181.388	1.00	40.000	142.000
281.000						
max						
instant	17379.000					
season	4.000					
yr	1.000					
mnth	12.000					
hr	23.000					
holiday	1.000					
weekday	6.000					
workingday	1.000					
weathersit	4.000					
temp	1.000					
atemp	1.000					
hum	1.000					
windspeed	0.851					
casual	367.000					
registered	886.000					
cnt	977.000					

Correlation Heatmap

```
# Correlation heatmap using day data
# Select columns
columns = ['season', 'mnth', 'holiday', 'weekday', 'workingday',
           'weathersit', 'temp', 'atemp', 'hum', 'windspeed', 'cnt']

# Plot correlation heatmap
fig, ax = plt.subplots(figsize=(10, 7))
sns.heatmap(data_day[columns].corr(), annot=True, fmt='.2f',
            cmap='coolwarm', center=0)
plt.tight_layout()
plt.show()
```



Distributions

```
# Plot distributions
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# daily rentals
data_day['cnt'].hist(ax=axes[0], bins=30)
axes[0].set_title('Distribution of daily rentals')
axes[0].set_xlabel('cnt')
```

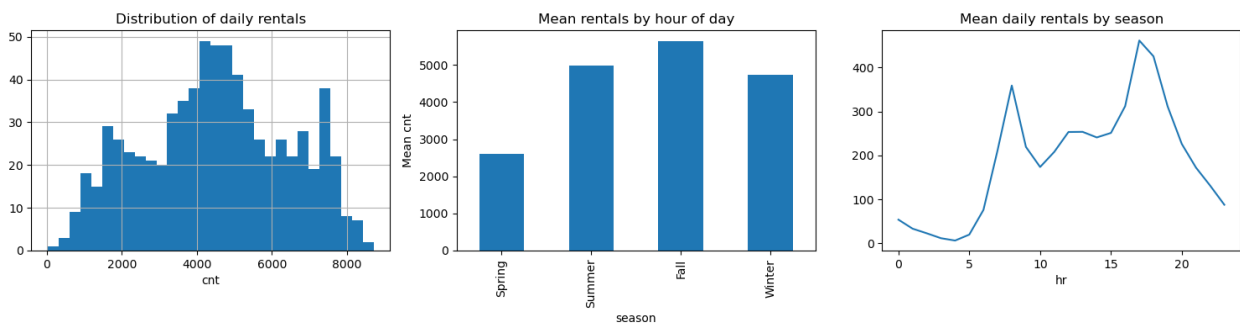
```

# daily rentals (hour)
data_hour.groupby('hr')['cnt'].mean().plot(ax=axes[2])
axes[1].set_title('Mean rentals by hour of day')
axes[1].set_xlabel('Hour')
axes[1].set_ylabel('Mean cnt')

# Seasons (mean)
season_labels = {1: 'Spring', 2: 'Summer', 3: 'Fall', 4: 'Winter'}
data_day.groupby('season')['cnt'].mean().rename(season_labels).plot(
    kind='bar', ax=axes[1])
axes[2].set_title('Mean daily rentals by season')

# Plot
plt.tight_layout()
plt.show()

```



Density

```

# KDE Density plots for selected continuous entries (normalised)
columns_of_int = ['temp', 'atemp', 'hum', 'windspeed', 'casual',
                  'registered', 'cnt']

# Plot Density
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# day
for col in columns_of_int:
    vals = data_day[col]
    sns.kdeplot((vals - vals.min()) / (vals.max() - vals.min()),
                ax=axes[0], label=col, fill=False)
axes[0].set_title('Feature densities (normalised) - day.csv')
axes[0].set_xlabel('Normalised value')
axes[0].legend(fontsize=8)

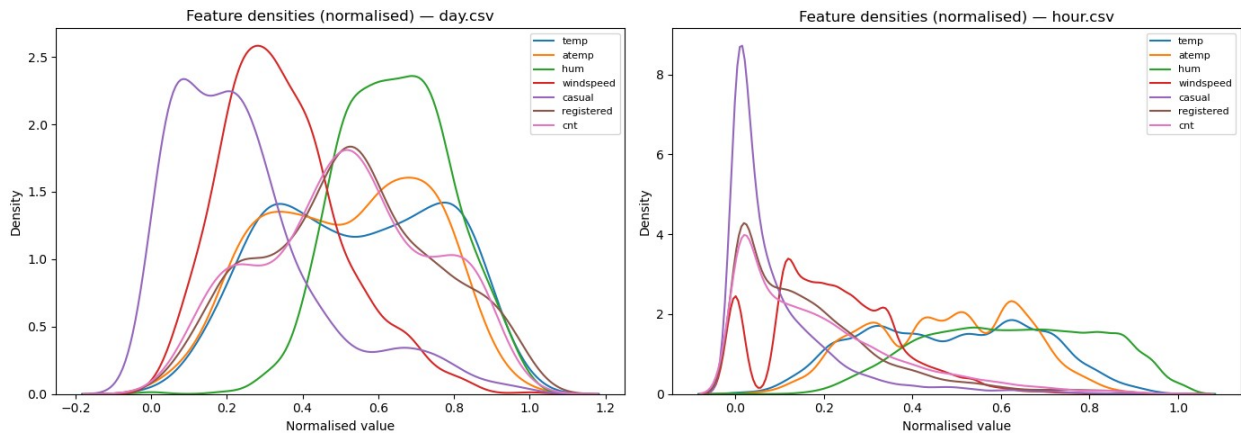
# hour
for col in columns_of_int:
    vals = data_hour[col]
    sns.kdeplot((vals - vals.min()) / (vals.max() - vals.min()),
                ax=axes[1], label=col, fill=False)

```



```
axes[1].set_title('Feature densities (normalised) – hour.csv')
axes[1].set_xlabel('Normalised value')
axes[1].legend(fontsize=8)
```

```
plt.tight_layout()
plt.show()
```



2. Module 1 — Vector-Space Analysis

Feature engineering

Selected based on the correlation heatmap and the task definition

```
# Define the features for clustering: Selected based on the
correlation heatmap
features = ['temp', 'hum', 'windspeed', 'weathersit', 'season',
'workingday', 'cnt']
```

2.1 Vector Representation

Feature vector: Each day is represented by 7 features selected based on the correlation heatmap and task definition.

Feature	Type	Intuition
temp	continuous (normalised)	Strong positive correlation with cnt
hum	continuous (normalised)	Moderate negative correlation with cnt
windspeed	continuous (normalised)	Mild negative effect on ridership
weathersit	ordinal (1–4)	Captures weather severity
season	ordinal (1–4)	Seasonal demand variation

Feature	Type	Intuition
workingday	binary (0/1)	Commuter vs leisure usage patterns
cnt	continuous (count)	Total demand: key signal

Preprocessing: All features are standardised to zero mean and unit variance (StandardScaler) so that features with different scales contribute equally to the Euclidean distance.

Distance measure: Euclidean distance (default for K-Means).

```
# Construct vector representation
X = StandardScaler().fit_transform(data_day[features]) # Replace with
actual feature matrix (standardised to zero mean and unit variance =
features with different scales contribute equally the Euclidean
distance)

# Print shape
print(f"Feature matrix shape: {X.shape}")

# Distance measure: We use euclidean distance (default for K-Means)
Feature matrix shape: (731, 7)
```

2.2 Clustering Method

Method 1: K-Means/K-Means++

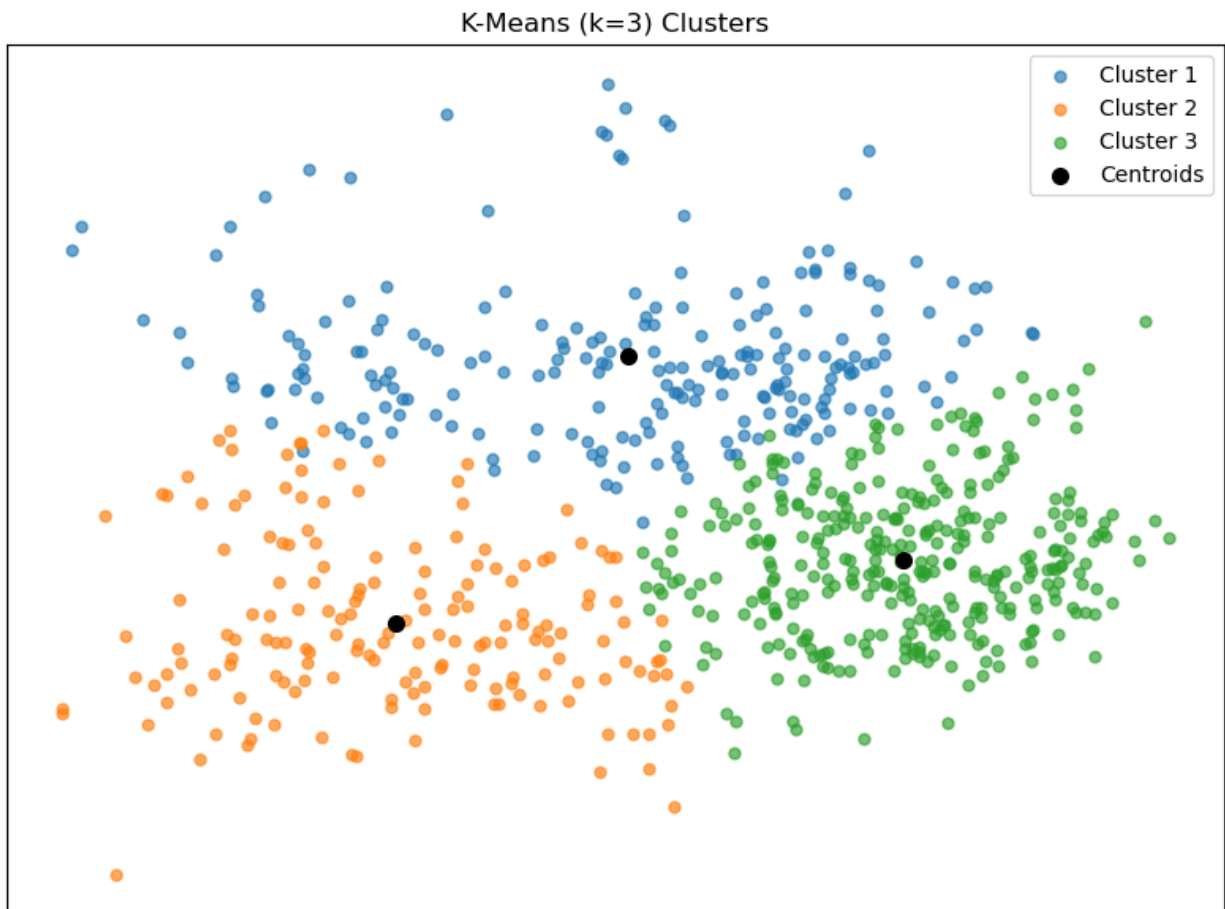
```
# Implement clustering algorithm here: Method 1 (K-means): Manual
without any parameter tuning
# Inititalize K and setup K-means (scikit learn)
K = 3
kmeans = KMeans(n_clusters=K, random_state=RANDOM_SEED, n_init=35)
labels = kmeans.fit_predict(X)

# Using PCA for 2D plots
pca = PCA(n_components=2, random_state=RANDOM_SEED).fit_transform(X)
cents = PCA(n_components=2,
random_state=RANDOM_SEED).fit(X).transform(kmeans.cluster_centers_)

# Plots clusters
fig, ax = plt.subplots(figsize=(8, 6))
for c in range(K):
    ax.scatter(*pca[labels == c].T, s=25, alpha=0.65, label=f'Cluster
{c+1}')
ax.scatter(*cents.T, s=49, marker='o', c='black', zorder=5,
label='Centroids')
ax.set(title=f'K-Means (k={K}) Clusters',)
ax.set_xticks([]); ax.set_yticks([])
```

```
ax.legend(); plt.tight_layout(); plt.show()

# Print cluster profiles (mean values of selected features)
print(data_day[features].assign(cluster=labels+1).groupby('cluster').mean().round(3))
```



	temp	hum	windspeed	weathersit	season	workingday
cnt						
cluster						
1	0.472	0.768	0.192	2.086	2.600	0.718
3764.468						
2	0.308	0.498	0.236	1.170	1.345	0.637
2898.292						
3	0.605	0.603	0.166	1.062	3.009	0.685
5790.847						

Parameter search

```
# grid search over K-Means & K-Means++ hyperparameters: Using
silhouette as evaluation method
```

```

# References:
# https://stackoverflow.com/questions/34611038/grid-search-for-hyperparameter-evaluation-of-clustering-in-scikit-learn
#
# https://scikit-learn.org/stable/modules/generated/sklearn.model\_selection.ParameterGrid.html#
#
# https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette\_score.html
# https://farshadabdulazeez.medium.com/understanding-silhouette-score-in-clustering-8aedc06ce9c4
h_params = {
    'n_clusters': range(2, 10),
    'init': ['k-means++', 'random'],
    'n_init': [10]
}

# Gridsearch
r = []
for p in ParameterGrid(h_params):
    kmeans = KMeans(random_state=RANDOM_SEED, **p)
    labels_k = kmeans.fit_predict(X)
    r.append({
        'k': p['n_clusters'],
        'init': p['init'],
        'silhouette': silhouette_score(X, labels_k),
        'inertia': kmeans.inertia_
    })

# Generate df with grid results
grid_results = pd.DataFrame(r).sort_values(
    ['silhouette', 'inertia'],
    ascending=[False, True]
).reset_index(drop=True)

# show results
display(grid_results)

# Print best
print(f"Best: k={int(grid_results.loc[0, 'k'])},
method='{grid_results.loc[0, 'init']}',
silhouette={float(grid_results.loc[0, 'silhouette']):.4f}")

```

	k	init	silhouette	inertia
0	2	k-means++	0.244613	3956.964456
1	2	random	0.244187	3956.969263
2	8	k-means++	0.242119	2107.353340
3	8	random	0.241977	2106.821046
4	9	k-means++	0.241298	1971.180281
5	9	random	0.239441	1972.560751

6	6	random	0.227104	2429.992502
7	3	random	0.226541	3280.320363
8	3	k-means++	0.226384	3280.229387
9	7	k-means++	0.225761	2261.193680
10	5	k-means++	0.225363	2623.783764
11	7	random	0.224880	2260.763688
12	5	random	0.223629	2624.558106
13	6	k-means++	0.220678	2420.634603
14	4	k-means++	0.217530	2901.400801
15	4	random	0.208710	2971.847961

Best: k=2, method='k-means++', silhouette=0.2446

The highest silhouette score is obtained at k=2 with k-means++, which seems to be a very coarse split. Second best segmentation k=8 with k-means++ is the strongest alternative and might be more informative.

```
# Implement clustering algorithm here: Method 1 (K-means): Manual
without any parameter tuning
# Initialize K and setup K-means (scikit learn)
k_values = [3, 4, 8]
init_opts = ['random', 'k-means++']

pca = PCA(n_components=2, random_state=RANDOM_SEED).fit(X)
pca_t = pca.transform(X)

# Use a fixed sample for cleaner visuals while keeping comparisons
fair
sample = np.random.default_rng(RANDOM_SEED)
sample_n = min(500, pca_t.shape[0])
sample_idx = sample.choice(pca_t.shape[0], size=sample_n,
replace=False)

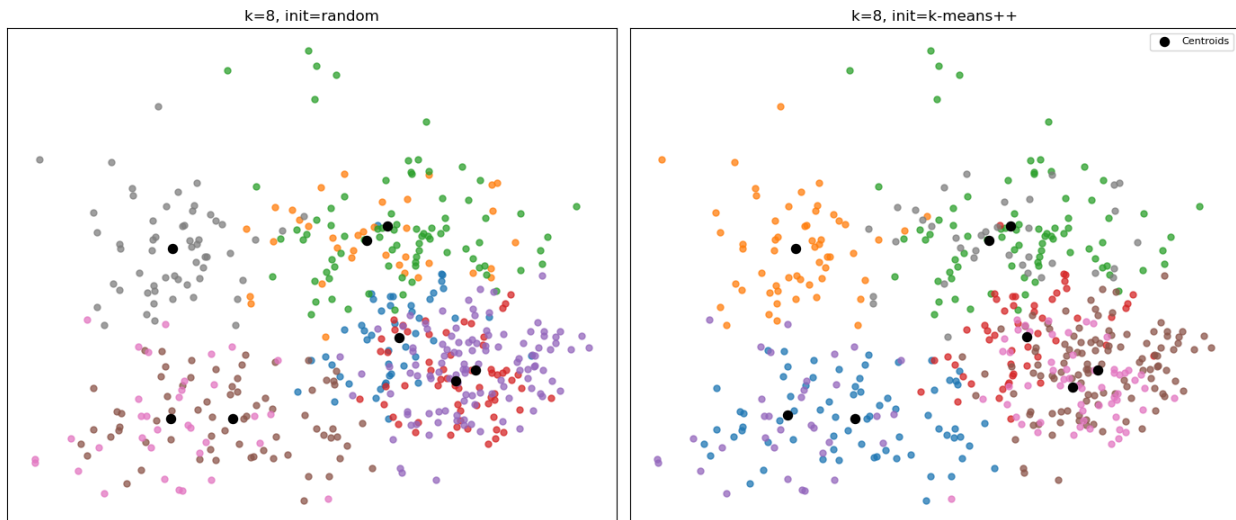
# Plots clusters: side-by-side figure per k value
for k in k_values:
    fig, axes = plt.subplots(1, 2, figsize=(14, 6), sharex=True,
sharey=True)
    for ax, init in zip(axes, init_opts):
        km = KMeans(n_clusters=k, init=init, random_state=RANDOM_SEED,
n_init=10)
        labels = km.fit_predict(X)
        cents = pca.transform(km.cluster_centers_)
        for c in range(k):
            m = (labels == c)
            m_sample = m & np.isin(np.arange(len(labels)), sample_idx)
            ax.scatter(*pca_t[m_sample].T, s=25, alpha=0.75)
            ax.scatter(*cents.T, s=52, marker='o', c='black', zorder=6,
label='Centroids')
        ax.set_title(f'k={k}, init={init}', fontsize=12)
```

```

ax.set_xticks([])
ax.set_yticks([])
axes[1].legend(loc='upper right', fontsize=8)
plt.tight_layout()
plt.show()

```





Method 1: Discussion

From the K-Means plots, `k=3` looks the cleanest and most interpretable, while `k=4` is a reasonable only for K-Means. `k=8` appears very over-segmented.

The K-Means clusters are somewhat usable for summarising broad daily patterns, but they are not strongly separated. Meaning that they are therefore likely most useful for interpretation, and not as strong evidence of naturally distinct clusters.

Method 2: DBSCAN

Feature engineering

```
# Select base features
features = ['temp', 'windspeed', 'workingday', 'casual', 'registered']

# Log transform count variables to reduce skewness
data_day['casual_log'] = np.log1p(data_day['casual'])
data_day['registered_log'] = np.log1p(data_day['registered'])

# Construct the feature matrix with transformed counts
X = data_day[['temp', 'windspeed', 'workingday', 'casual_log',
              'registered_log']]

# Standardize features so each contributes equally to distance
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Plot the k-distance graph to choose epsilon for DBSCAN

```
# Plot the k-distance graph to choose eps for DBSCAN
# Set min_samples as twice the number of features
min_samples_estimate = 2 * X.shape[1]
```

```

neighbors = NearestNeighbors(n_neighbors=int(min_samples_estimate))
neighbors_fit = neighbors.fit(X_scaled)
distances, indices = neighbors_fit.kneighbors(X_scaled)
distances = np.sort(distances[:, int(min_samples_estimate) - 1])

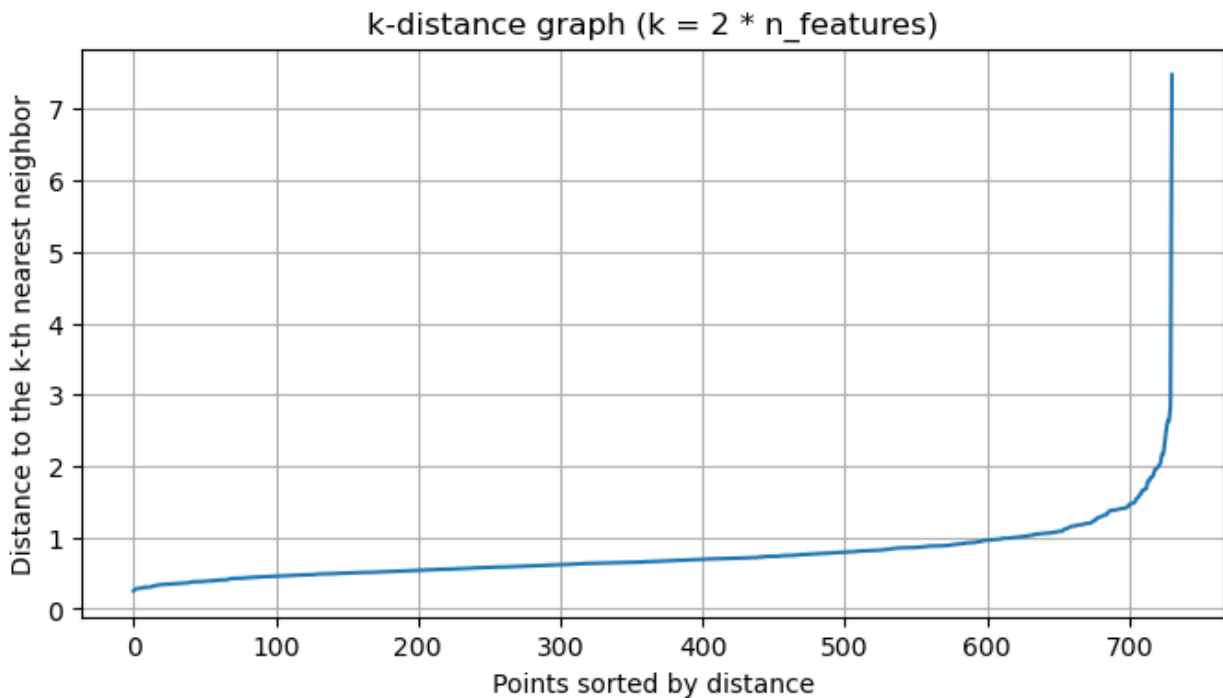
```

```

plt.figure(figsize=(8, 4))
plt.plot(distances)
plt.title('k-distance graph (k = 2 * n_features)')
plt.xlabel('Points sorted by distance')
plt.ylabel('Distance to the k-th nearest neighbor')
plt.grid(True)
plt.show()

```

*# After plotting, inspect the curve for an elbow point. This value of distance
will be used as the eps parameter for DBSCAN.*



```

# Perform t-SNE to visualise potential clusters
# Use a moderate perplexity. t-SNE is stochastic; random_state
# ensures reproducibility.
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
X_tsne = tsne.fit_transform(X_scaled)

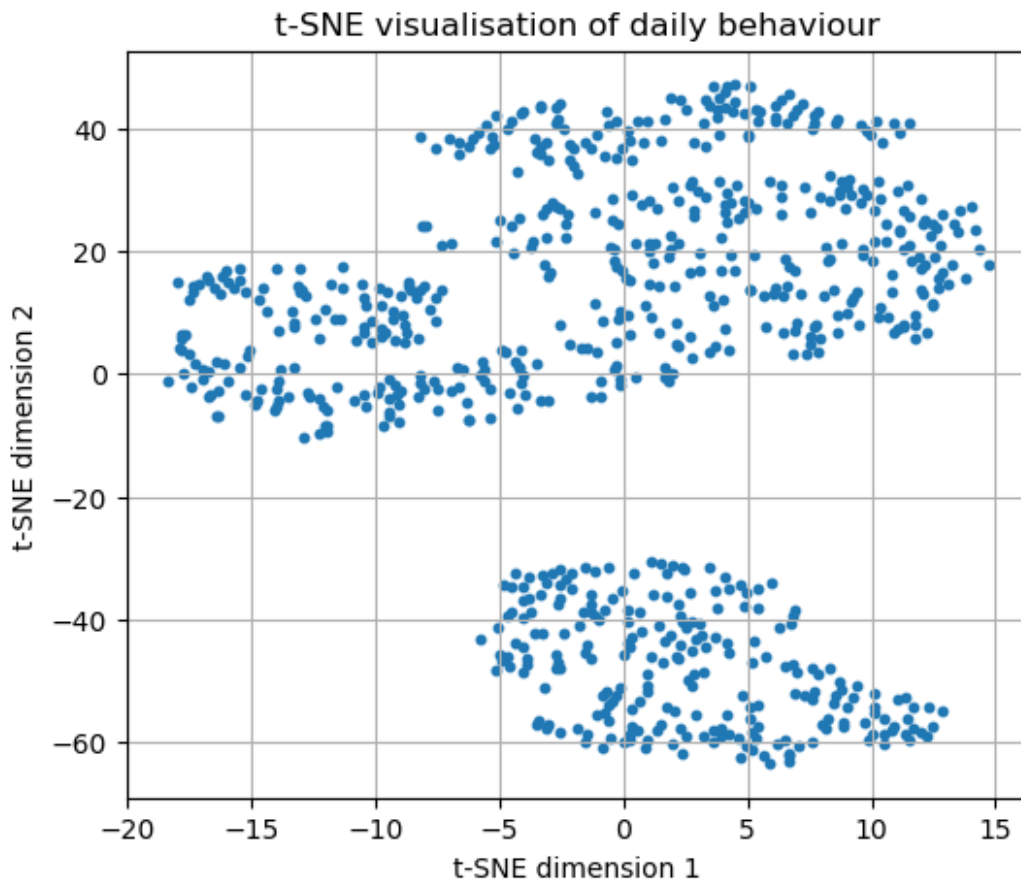
plt.figure(figsize=(6, 5))
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], s=10)
plt.title('t-SNE visualisation of daily behaviour')

```



```
plt.xlabel('t-SNE dimension 1')
plt.ylabel('t-SNE dimension 2')
plt.grid(True)
plt.show()
```

Look for visually separated clusters. If distinct clouds appear, DBSCAN is likely appropriate.



Choosing DBSCAN parameters

The DBSCAN algorithm requires two key parameters:

- **eps (epsilon)** – defines the radius of the neighborhood for clustering. Use the elbow point from the k-distance graph as a guideline for choosing **eps**. The epsilon parameter was selected from the k-distance graph by identifying the elbow point at approximately 1.4, which represents the transition from dense regions to sparse regions.
- **min_samples** – minimum number of points required to form a dense region. A common heuristic is to set **min_samples** to twice the number of features; in this case, that is 10. You may adjust it based on domain knowledge.

After examining the k-distance plot, set **eps** in the next cell and run the clustering. Note that changing **eps** or **min_samples** will affect the number and composition of clusters.

```

# Apply DBSCAN clustering
from sklearn.cluster import DBSCAN

# TODO: Replace the value below with your chosen eps from the k-
distance graph.
eps_value = 1.4 # <-- Set your chosen epsilon here
min_samples_value = int(2 * X.shape[1]) # default to 2 * n_features

# Fit DBSCAN
dbscan = DBSCAN(eps=eps_value, min_samples=min_samples_value)
labels = dbscan.fit_predict(X_scaled)

# Assign cluster labels back to the data
data_day['cluster'] = labels

# Summarise cluster sizes (including noise labelled as -1)
cluster_counts = data_day['cluster'].value_counts()
print('Cluster counts (including noise):')
print(cluster_counts)

# Compute mean values of key variables per cluster
summary = data_day.groupby('cluster')[['temp', 'windspeed', 'casual',
'registered']].mean()
print('Cluster summaries (mean values):')
print(summary)

```

```

Cluster counts (including noise):
1      494
0      226
-1       11
Name: cluster, dtype: int64
Cluster summaries (mean values):

```

	temp	windspeed	casual	registered
cluster				
-1	0.285725	0.301096	107.727273	686.000000
0	0.484783	0.188015	1397.300885	3003.323009
1	0.504904	0.189154	613.445344	4020.981781

Method 2: Discussion

DBSCAN identified a small set of anomalous days characterized by low temperature and low rental activity.

Cluster 0 (226 days) temp → moderate (0.48)

casual → very high (1397)

registered → moderate

This cluster represents leisure-oriented usage patterns.

Cluster 1 (494 days) temp → similar (0.50)

casual → moderate

registered → very high (4020)

This cluster corresponds to regular commuter-driven demand.

3. Module 2 — Graph-Based Analysis

3.1 Graph Representation and Modeling Choices

The data are represented as a day-level similarity graph using the daily dataset component `day.csv`.

- **Nodes:** Each node represents a single calendar day from `day.csv` (731 nodes). Such that we can identify recurring day types.
- **Edges:** The hypothesis is that days with similar context should group together; therefore, an edge connects two days with similar contextual conditions.
- **Graph type:** The graph is weighted and undirected. A weighted graph is selected because days with similar weather and calendar conditions are not necessarily equally similar, so edge weights preserve some similarity strength. The graph is undirected because similarity between days are symmetric, making a directional-graph unnecessary.

Feature-space for edge construction

- Weather and seasonality: weather situation, temperature, humidity, wind speed, season, month
- Calendar context: workdays, holidays, weekdays, or potentially weekends

Target Variables

- **total rentals**, **casual riders**, and **registered riders** are not used to construct edges because they are used as target variables to explain any differences across groups; if included, communities would be partially predefined. Instead they are attached as node attributes (e.g., used for community profiling)

Construction rule

- Standardize contextual features, since variables are measured on different scales; without scaling, large-scale variables dominate distance calculations.
- compute k-nearest neighbors using Euclidean distance in the standardized contextual feature space.
- set $k = 10$ to maintain a sparse local-neighborhood graph and avoid a fully connected noisy network.
- assign weighted edges from a Gaussian-kernel similarity transformation of neighbor distances, so larger weights indicate stronger contextual similarity.

This representation supports identification of recurring day types from shared context, followed by behavioral interpretation using demand and rider-composition attributes.

Graph Construction

Define the feature space used for similarity and applies standardization before graph assembly

```
import networkx as nx

# Contextual feature space
features = [
    "weathersit", "temp", "hum", "windspeed", "season", "mnth",
    "workingday", "holiday", "weekday"
]

X_context = data_day[features].copy()
scaler_graph = StandardScaler()
X_context_scaled = scaler_graph.fit_transform(X_context)
```

Compute nearest neighbors in the standardised contextual space and convert distances into a similarity scale for the weighted edges

```
# kNN search in the standardised contextual space
k_n = 10
knn_graph = NearestNeighbors(n_neighbors=k_n + 1, metric="euclidean")
knn_graph.fit(X_context_scaled)
knn_graph_distances, knn_graph_indices =
knn_graph.kneighbors(X_context_scaled)

# Scaling: Distance to similarity (Gaussian kernel parameter)
knn_non_dist = knn_graph_distances[:, 1:].ravel()
sigma = float(np.mean(knn_non_dist)) if np.mean(knn_non_dist) > 0 else
1.0
```

Build the weighted & undirected graph by adding day nodes and linking the context-similar neighbors with similarity-based edge weights

```
# Build weighted undirected graph
G = nx.Graph()

# Add day nodes with metadata for target vars
for i, row in data_day.reset_index(drop=True).iterrows():
    node_attrs = {
        "day_id": int(i),
        "dteday": row["dteday"] if "dteday" in data_day.columns else
str(i),
        "cnt": float(row["cnt"]),
        "casual": float(row["casual"]),
        "registered": float(row["registered"]),
    }
    G.add_node(i, **node_attrs)
```

```

# Add the weighted edges from kNN neighborhoods
for i in range(len(data_day)):
    for pos in range(1, k_n + 1):
        j = int(knn_graph_indices[i, pos])
        d = float(knn_graph_distances[i, pos])
        w = float(np.exp(-(d ** 2) / (2 * (sigma ** 2))))

        if i == j:
            continue

        if G.has_edge(i, j):
            if w > G[i][j]["weight"]:
                G[i][j]["weight"] = w
        else:
            G.add_edge(i, j, weight=w)

```

Print basic graph representation statistics

```

# Basic graph statistics
weights = np.array([d["weight"] for _, _, d in G.edges(data=True)],
dtype=float)
components = list(nx.connected_components(G))

print(f"Nodes: {G.number_of_nodes()}")
print(f"Edges: {G.number_of_edges()}")
print(f"Connected components: {len(components)}")
print(f"Largest component size: {max(len(c) for c in components)}")
print(f"Average degree: {np.mean([deg for _, deg in
G.degree()]):.2f}")
print(f"Edge weight range: [{weights.min():.4f},
{weights.max():.4f}]")

Nodes: 731
Edges: 4743
Connected components: 2
Largest component size: 710
Average degree: 12.98
Edge weight range: [0.0013, 0.9981]

```

Graph Visualisation: Using largest connected components

```

# Graph plot
largest_nodes = max(nx.connected_components(G), key=len)
G_largest_cc = G.subgraph(largest_nodes).copy()
pos = nx.spring_layout(G_largest_cc, seed=RANDOM_SEED, k=0.15)

# Plot
plt.figure(figsize=(11, 8))
weights_plot = [edata["weight"] for _, _, edata in

```

```

G_largest_cc.edges(data=True)]

nx.draw_networkx_nodes(
    G_largest_cc,
    pos,
    node_size=14,
    node_color="steelblue",
    alpha=0.85
)

nx.draw_networkx_edges(
    G_largest_cc,
    pos,
    width=[0.5 + 1.8 * w for w in weights_plot],
    edge_color=weights_plot,
    edge_cmap=plt.cm.Blues,
    alpha=0.35
)

plt.title("Day-level Similarity Graph (Largest Connected Component)")
plt.axis("off")
plt.tight_layout()
plt.show()

/var/folders/zc/7mltnf3d7g9_16f_w0n9dlcc0000gn/T/
ipykernel_28138/4045861617.py:18: DeprecationWarning: `alltrue` is
deprecated as of NumPy 1.25.0, and will be removed in NumPy 2.0.
Please use `all` instead.
    nx.draw_networkx_edges(

```

Day-level Similarity Graph (Largest Connected Component)



3.2 Graph Mining Methods

Experiment 1: Community detection **Experiment 2:** Spectral Clustering **Experiment 3:** Graph Embeddings

Experiment 1: Community Detection (Louvain)

Experiment 1 uses applies Louvain community detection on the generated graph-representation

```
import networkx.algorithms.community as nx_comm

# Louvain community detection
res = 1.2 # Resolution param to control community size: Tested 0.8, 1.0, 1.2
partition = nx_comm.louvain_communities(
    G_largest_cc,
    weight="weight",
    resolution=res,
    seed=RANDOM_SEED
)

# Modularity value for the partition
modularity_val = nx_comm.modularity(
```

```

    G_largest_cc,
    partition,
    weight="weight"
)

# Map each node to its community label
node_to_comm = {}
for cid, nodes in enumerate(partition):
    for n in nodes:
        node_to_comm[n] = cid

print(f"Resolution: {partition}")
print(f"Number of communities: {len(partition)}")
print(f"Modularity: {modularity_val:.4f}")

Resolution: [{384, 387, 6, 390, 391, 10, 397, 17, 18, 19, 403, 405,
24, 409, 25, 27, 411, 30, 31, 32, 34, 419, 39, 424, 426, 54, 55, 440,
62, 67, 68, 69, 454, 73, 74, 81, 82, 88, 89, 90, 97, 102, 111, 368,
373, 375, 376}, {2, 3, 388, 5, 4, 389, 9, 394, 395, 11, 13, 398, 12,
401, 402, 404, 20, 23, 408, 26, 410, 416, 33, 37, 422, 423, 40, 41,
38, 44, 429, 430, 46, 45, 431, 432, 433, 436, 52, 53, 59, 60, 61, 65,
66, 451, 450, 72, 464, 86, 87, 93, 367, 369, 370, 374, 377, 382, 383},
{64, 449, 386, 1, 323, 708, 134, 645, 456, 267, 715, 400, 274, 596,
155, 120, 477, 603, 225, 610, 99, 169, 491, 113, 183, 568, 442, 701},
{514, 516, 520, 522, 523, 524, 529, 530, 531, 535, 536, 47, 48, 75,
76, 80, 83, 95, 96, 103, 107, 109, 110, 114, 115, 118, 124, 125, 128,
129, 130, 131, 135, 139, 396, 144, 145, 146, 150, 152, 153, 156, 157,
158, 412, 160, 417, 418, 163, 164, 165, 167, 425, 437, 438, 439, 443,
444, 446, 452, 453, 457, 458, 459, 460, 461, 465, 466, 467, 468, 472,
474, 475, 479, 480, 482, 487, 489, 495, 496, 501, 502, 503, 509, 510},
{0, 385, 259, 133, 266, 399, 273, 406, 665, 35, 427, 301, 560, 567,
441, 700, 63, 448, 707, 455, 588, 728, 217, 602, 91, 224, 609, 98,
483, 616, 105, 490, 238, 112}, {256, 537, 538, 541, 542, 543, 544,
545, 548, 549, 551, 552, 557, 558, 562, 563, 564, 565, 569, 570, 571,
572, 573, 576, 577, 578, 579, 586, 590, 591, 592, 593, 594, 598, 599,
600, 604, 605, 606, 607, 608, 612, 613, 615, 618, 619, 620, 621, 622,
628, 629, 643, 172, 174, 178, 179, 180, 181, 185, 186, 187, 191, 192,
193, 194, 195, 198, 199, 200, 202, 205, 206, 207, 208, 209, 212, 213,
216, 219, 220, 221, 222, 223, 226, 227, 228, 229, 233, 234, 235, 237,
240, 241, 242, 243, 254, 255}, {512, 260, 519, 652, 141, 526, 659,
148, 533, 281, 540, 288, 673, 162, 547, 295, 680, 554, 302, 687, 176,
561, 309, 694, 316, 190, 575, 197, 582, 330, 204, 589, 337, 722, 211,
470, 344, 218, 351, 484, 358, 232, 617, 239, 624, 498, 246, 631, 505,
253, 638, 127}, {257, 258, 517, 261, 262, 263, 264, 265, 268, 269,
270, 275, 283, 285, 286, 290, 298, 299, 300, 555, 556, 559, 307, 566,
313, 318, 320, 580, 326, 583, 584, 585, 587, 332, 338, 597, 601, 347,
348, 349, 353, 614, 625, 635, 636, 639, 641, 642, 647, 653, 656, 657,
663, 664, 666, 668, 669, 670, 159, 676, 682, 171, 684, 173, 688, 689,
177, 696, 188, 706, 709, 710, 711, 201, 716, 719, 214, 215, 230, 236,
244, 251}, {648, 649, 650, 654, 655, 271, 272, 660, 661, 278, 662,

```



```

279, 276, 277, 671, 289, 674, 675, 292, 293, 678, 677, 296, 297, 683,
685, 303, 304, 305, 690, 306, 692, 310, 311, 695, 312, 697, 699, 698,
317, 702, 703, 704, 705, 321, 328, 713, 712, 331, 717, 333, 335, 334,
718, 341, 342, 345, 346, 352, 627, 632, 633, 634}, {640, 324, 325,
720, 723, 339, 725, 726, 727, 340, 730, 667, 284, 354, 355, 356, 291,
360, 361, 362, 363, 248, 249, 250, 319}, {515, 132, 136, 521, 138,
137, 142, 527, 528, 143, 534, 151, 166, 170, 58, 445, 447, 79, 473,
94, 478, 481, 100, 485, 486, 101, 488, 122, 123, 492, 108, 494, 493,
626, 499, 500, 116, 117, 121, 506, 507, 508, 381}, {71, 8, 393, 78,
15, 463, 85, 22, 407, 729, 92, 29, 414, 36, 421, 106, 43, 428, 365,
50, 435, 372, 57, 379}, {644, 518, 7, 392, 140, 525, 14, 147, 532, 21,
154, 539, 28, 413, 161, 546, 420, 168, 553, 42, 175, 49, 434, 182, 56,
189, 574, 196, 581, 70, 203, 77, 462, 721, 210, 595, 84, 469, 476,
357, 231, 364, 623, 497, 371, 245, 630, 119, 504, 378, 252, 126, 511},
{322, 329, 714, 651, 336, 658, 343, 280, 350, 287, 672, 294, 679, 686,
308, 693, 315, 637}]
Number of communities: 14
Modularity: 0.8206

```

The Louvain partition at resolution of 0.8, 1.0, and 1.2 shows high modularities (0.8159, 0.8177, and 0.8206 respectively), indicating a strong structural coherence in the graph-representation.

Community Profiles

Behavioral and contextual profiling for the detected communities, used to inspect behavioral separation and interpretability.

```

# Behavioral and contextual profiling by community
profile_df = data_day.loc[list(G_largest_cc.nodes())].copy()
profile_df["community"] = profile_df.index.map(node_to_comm)

# Behavioral profile: mean, median, std, count for cnt, casual,
registered
behavior_profile = (
    profile_df
    .groupby("community")[["cnt", "casual", "registered"]]
    .agg(["mean", "median", "std", "count"])
    .round(2)
)

# Context profile: mean values for contextual features
context_profile = (
    profile_df
    .groupby("community")[["temp", "hum", "windspeed", "weathersit",
"season", "workingday", "holiday", "weekday"]]
    .mean()
    .round(3)
)

```

```
# Print dataframes
print("Behavioral profile by community:")
display(behavior_profile)
print("\nContext profile by community:")
display(context_profile)
```

Behavioral profile by community:

\	cnt			count	casual		
	mean	median	std		mean	median	std
count							
community							
0	2274.66	1977.0	1063.13	47	170.23	148.0	137.17
47							
1	2900.20	2425.0	1375.69	59	232.69	163.0	202.50
59							
2	4049.96	4370.0	1659.42	28	1253.89	1273.0	777.76
28							
3	5401.76	5312.0	1403.13	85	856.26	834.0	315.26
85							
4	3951.15	4108.0	2171.55	34	1192.50	1076.5	860.52
34							
5	5967.06	5895.0	1353.15	97	962.74	949.0	230.27
97							
6	5132.35	5043.5	1666.02	52	1687.12	1725.0	739.20
52							
7	4930.62	4669.0	1507.07	82	602.85	536.5	338.04
82							
8	5204.87	5146.0	1385.14	63	543.87	471.0	288.82
63							
9	2211.28	2416.0	1020.84	25	208.28	204.0	146.67
25							
10	4497.51	4367.0	1498.73	43	639.07	630.0	376.30
43							
11	2470.29	2356.5	1163.31	24	695.17	578.5	565.36
24							
12	4828.04	5119.0	2291.72	53	1663.96	1782.0	985.78
53							
13	4865.67	4677.5	1799.46	18	1395.39	1262.0	755.68
18							

community	registered			count
	mean	median	std	
0	2104.43	1769.0	970.20	47
1	2667.51	2147.0	1237.97	59
2	2796.07	2879.0	1023.44	28
3	4545.51	4653.0	1171.00	85

4	2758.65	2709.5	1383.80	34
5	5004.32	4841.0	1187.27	97
6	3445.23	3302.0	1058.26	52
7	4327.77	3921.0	1259.74	82
8	4661.00	4531.0	1266.30	63
9	2003.00	2169.0	933.07	25
10	3858.44	3729.0	1227.13	43
11	1775.12	1527.0	749.18	24
12	3164.08	3312.0	1373.64	53
13	3470.28	3041.0	1185.15	18

Context profile by community:

	temp	hum	windspeed	weathersit	season	workingday
holiday \ community						
0	0.290	0.692	0.186	2.064	1.191	1.000
0.0						
1	0.283	0.489	0.234	1.017	1.102	1.000
0.0						
2	0.494	0.783	0.169	2.036	2.536	0.000
0.0						
3	0.553	0.566	0.203	1.000	1.835	1.000
0.0						
4	0.468	0.780	0.197	2.118	2.382	0.000
0.0						
5	0.721	0.587	0.166	1.000	3.010	1.000
0.0						
6	0.566	0.610	0.173	1.000	3.038	0.000
0.0						
7	0.543	0.724	0.173	2.000	3.585	0.988
0.0						
8	0.407	0.610	0.165	1.000	3.984	1.000
0.0						
9	0.401	0.792	0.208	2.360	2.320	1.000
0.0						
10	0.559	0.741	0.221	2.000	1.977	1.000
0.0						
11	0.292	0.482	0.236	1.083	1.167	0.000
0.0						
12	0.527	0.536	0.204	1.038	2.019	0.000
0.0						
13	0.375	0.561	0.175	1.111	4.000	0.000
0.0						
	weekday					
community						
0	3.617					

1	2.559
2	0.000
3	3.388
4	6.000
5	3.010
6	0.000
7	3.159
8	3.032
9	2.960
10	2.279
11	0.000
12	6.000
13	6.000

Communities show clear differences in demand level and rider composition (cnt, casual, registered) together with some distinct contextual averages, supporting both behavioral differentiation and practical interpretability.

Communities Assignments Visualisation

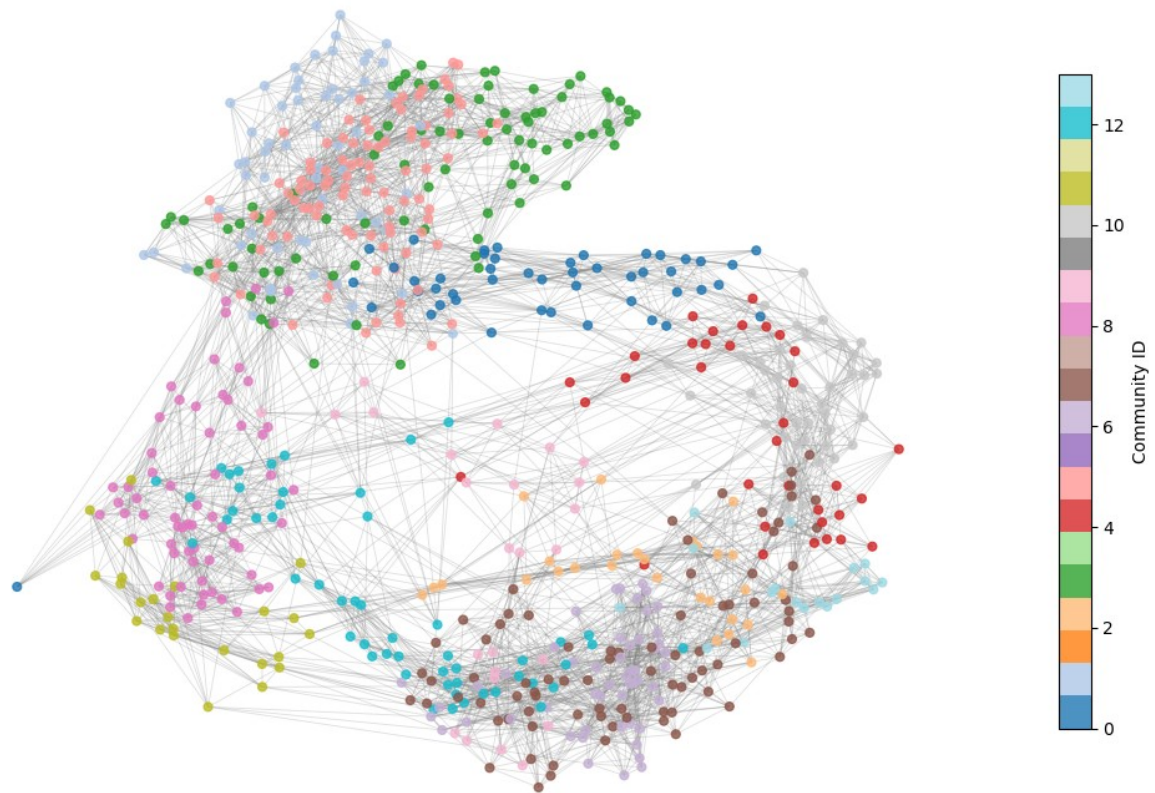
```
# Community assignment visualization (primary partition)
node_list = list(G_largest_cc.nodes())
community_ids = [node_to_comm[n] for n in node_list]

# Plot
plt.figure(figsize=(11, 8))
nx.draw_networkx_edges(
    G_largest_cc,
    pos,
    edge_color="gray",
    alpha=0.25,
    width=0.6
)

nodes = nx.draw_networkx_nodes(
    G_largest_cc,
    pos,
    nodelist=node_list,
    node_color=community_ids,
    cmap=plt.cm.tab20,
    node_size=24,
    alpha=0.8
)

cbar = plt.colorbar(nodes, shrink=0.7)
cbar.set_label("Community ID")
plt.title("Community Assignments")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Community Assignments



The community-assignment plot provides visual support for the Louvain partition. Several color groups occupy coherent regions with relatively few links between them.

Experiment 1: Discussion

Louvain community detection on the largest connected component identifies stable community structures across tested all resolutions (11 communities at 0.8, 13 at 1.0, and 14 at 1.2), with consistently high modularity (0.8159-0.8206). This supports **H1**, indicating that days with similar weather and calendar context forms coherent graph communities.

The behavioral profiles across communities show a clear differences in the total demand (`cnt`) and rider composition (`casual`, `registered`), supporting **H2**. Several communities are registered-dominated with high demand, while others show higher casual activity and lower registered counts.

Furthermore, the context profiles indicate interpretable regimes; calendar (seasonality, workday, weekend) and weather severity differences, supporting **H3** and suggesting that communities correspond to meaningful day types rather than just arbitrary partitions.

Experiment 1: Limitations and Improvements

Limitations

- Community labels are unlabeled IDs, which should be matched to the specific target variables.
- Results depend on graph construction choices (k , similarity kernel, and selected features).
- Analysis is performed on the largest connected components, so a small set of days outside that component is not represented.

Potential Improvements

- Add quantitative partition-stability checks across random seeds/resolution values.
- Compare Louvain with an alternative community algorithm (for example, Leiden) to test robustness of findings.

Experiment 2: Spectral Clustering

K-way spectral clustering is used on the generated graph representation for Experiment 2; to determine the optimal number of k partitions, the eigengap method is used.

Optimal Number of K

The eigengap method identifies the optimal number of clusters by finding the largest numerical jump between consecutive eigenvalues of the graph Laplacian.

It works by finding a clear break or 'gap' in the graph's matrix values that separates the actual community structure from the background data.

```
# the affinity matrix
aff_mx = nx.to_numpy_array(G_largest_cc, weight='weight')
# Normalised Laplacian  $L_{sym} = I - D^{-1/2} A D^{-1/2}$ 
degree = aff_mx.sum(axis=1)
D_inv_sqrt = np.diag(1.0 / np.sqrt(degree))
L_sym = np.eye(len(degree)) - D_inv_sqrt @ aff_mx @ D_inv_sqrt
# the eigenvalues of the normalised Laplacian
eigenvalues, _ = np.linalg.eigh(L_sym)

# largest eigengap for a range of K
gaps = []
k_range = range(2, 11)

for k in k_range:
    gap = eigenvalues[k] - eigenvalues[k - 1]
    gaps.append(gap)

optimal_k = k_range[np.argmax(gaps)]

print(f"The optimal number of communities based on Eigengap is:
{optimal_k}\n")
```

The optimal number of communities based on Eigengap is: 10

K-way Spectral Clustering

After determining the optimal number of communities, let's proceed with the division of the data into `k` partitions.

```
spectral_model = SpectralClustering(
    n_clusters=optimal_k,
    affinity='precomputed',
    assign_labels='kmeans',
    random_state=42
)
lab = spectral_model.fit_predict(aff_mx)
pos = nx.spring_layout(G_largest_cc, seed=42, k=0.15)
# dictionary assigning node to each community
node_to_community = dict(zip(G_largest_cc.nodes(), lab))
# node colors for plotting
node_colors = [node_to_community[node] for node in
G_largest_cc.nodes()]
```

Cluster Profiles

The behavioral and contextual profiles of the different partitions are inspected to check for actual differences.

```
print("--- Cluster Profiling ---")

# dictionary mapping the node IDs to their new community labels
node_to_community = dict(zip(G_largest_cc.nodes(), lab))

# Subset dataframe for just the nodes in our main graph
prof = data_day.loc[list(G_largest_cc.nodes())].copy()
# Add the cluster labels
prof["cluster"] = prof.index.map(node_to_community)

b_prof = (
    prof
    .groupby("cluster")[["cnt", "casual", "registered"]]
    .agg(["mean", "median", "std", "count"])
    .round(2)
)

c_prof = (
    prof
    .groupby("cluster")[["temp", "hum", "windspeed", "weathersit",
"season", "workingday", "holiday", "weekday"]]
    .mean()
```

```

        .round(3)
    )

    print(f"\nBehavioral profile by cluster (cluster={optimal_k}):")
    display(b_prof)

    print(f"\nContext profile by cluster (cluster={optimal_k}):")
    display(c_prof)

```

--- Cluster Profiling ---

Behavioral profile by cluster (cluster=10):

\	cnt				casual			
	mean	median	std	count	mean	median	std	count
cluster								
0	5085.61	5087.0	1311.13	59	529.51	433.0	292.61	59
1	5871.15	5859.0	1416.04	144	936.57	905.5	255.92	144
2	4947.25	4744.0	1549.60	71	1597.96	1582.0	723.51	71
3	3318.31	3194.0	1694.07	91	390.97	247.0	362.08	91
4	2728.59	2409.5	1589.66	34	796.06	578.5	711.25	34
5	3822.95	3830.0	1757.34	105	468.73	325.0	382.15	105
6	4391.46	4128.0	1819.41	101	518.64	449.0	348.20	101
7	3550.61	3409.0	2183.31	31	1091.35	956.0	881.54	31
8	4914.28	5119.0	2190.44	50	1697.38	1794.5	947.40	50
9	5084.33	5164.5	1856.72	24	1464.62	1262.0	809.29	24

cluster	registered			
	mean	median	std	count
0	4556.10	4372.0	1201.50	59
1	4934.58	4850.0	1227.44	144
2	3349.30	3221.0	976.12	71
3	2927.34	2931.0	1400.00	91
4	1932.53	1657.5	983.13	34
5	3354.22	3614.0	1455.95	105
6	3872.81	3767.0	1542.71	101
7	2459.26	2507.0	1347.47	31

8	3216.90	3329.5	1317.91	50
9	3619.71	3482.5	1186.24	24

Context profile by cluster (cluster=10):

holiday \ cluster	temp	hum	windspeed	weathersit	season	workingday
0 0.0	0.394	0.608	0.165	1.000	4.000	1.0
1 0.0	0.684	0.607	0.167	1.000	2.694	1.0
2 0.0	0.563	0.656	0.170	1.282	3.070	0.0
3 0.0	0.414	0.711	0.205	2.033	1.538	1.0
4 0.0	0.317	0.569	0.228	1.353	1.265	0.0
5 0.0	0.369	0.489	0.234	1.010	1.371	1.0
6 0.0	0.524	0.750	0.176	2.129	3.416	1.0
7 0.0	0.455	0.759	0.205	2.129	2.032	0.0
8 0.0	0.540	0.534	0.202	1.000	2.020	0.0
9 0.0	0.397	0.617	0.174	1.292	4.000	0.0

cluster	weekday
0	3.017
1	3.125
2	0.000
3	3.022
4	0.000
5	2.962
6	3.059
7	6.000
8	6.000
9	6.000

The different partitions show coherent results with **H2**; clusters have distinct contextual profiles. Moreover, the highest average demand for the service (**cnt**) is consistent with some contextual variables; for instance, **temp** in partition 1 is the highest, as is the average number of rides demanded. This connection between peak demand and optimal weather conditions provides the practical interpretation required to support H3.

Clusters Assignment Visualization

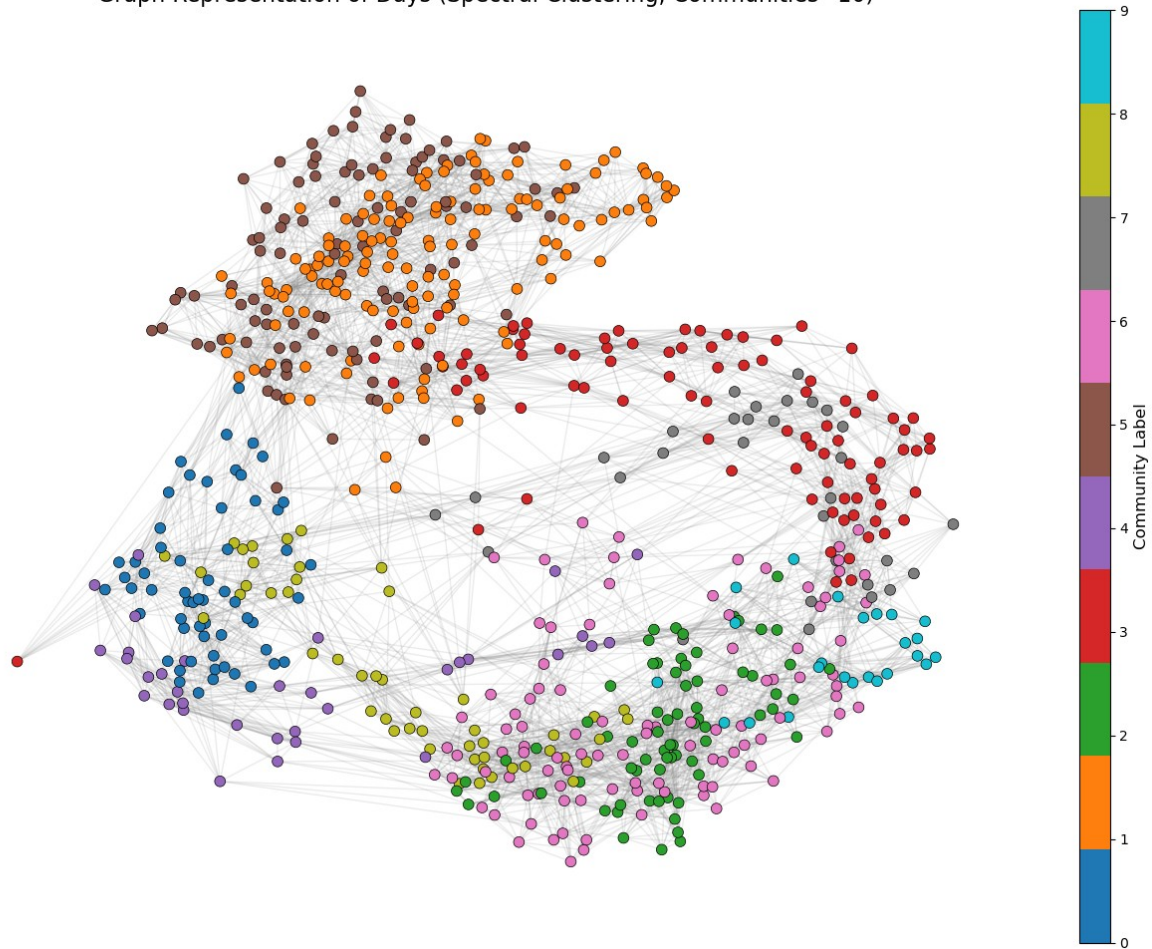
```
plt.figure(figsize=(14, 10))
plt.title(f"Graph Representation of Days (Spectral Clustering,  
Communities={optimal_k})", fontsize=16)

nx.draw_networkx_edges(G_largest_cc, pos, alpha=0.15,  
edge_color="gray")
# Draw nodes distinguished by community
scatter = nx.draw_networkx_nodes(  
    G_largest_cc,  
    pos,  
    node_color=node_colors,  
    cmap=plt.cm.tab10,  
    node_size=60,  
    edgecolors="black",  
    linewidths=0.5  
)

colorbar = plt.colorbar(scatter, ticks=range(optimal_k), pad=0.02,  
aspect=30)  
colorbar.set_label('Community Label', fontsize=12)

plt.axis("off")  
plt.tight_layout()  
plt.show()
```

Graph Representation of Days (Spectral Clustering, Communities=10)



Each node is a day and is colored by its cluster assignment. Spatially coherent regions of the same color indicate clusters that occupy localized regions of the graph, supporting H1.

Experiment 2: Discussion

Spectral Clustering on the largest connected component identifies k structurally coherent clusters via the eigengap.

Looking at the behavioral data, it's clear that each cluster has a unique signature in terms of volume and user composition. This confirms **H2**, as the clusters separate commuter-heavy days from leisure ones. The contextual variables—temperature, season, and whether it's a workday—make these groups easy to interpret, supporting **H3**.

From a geometric point of view, the separation seen in the graph provides solid evidence for **H1**. While Louvain (Example 1) focuses on local density, Spectral Clustering looks at the global graph structure. Despite these differences, both experiments show consistent results and strongly support our initial hypotheses.

Experiment 2: Limitations and Improvements

Limitations

- Spectral Clustering requires the full adjacency matrix in memory, which implies a heavier computational cost ($O(n^2)$) and limits the size of data;
- The eigengap can be ambiguous when multiple gaps are of similar magnitude; the choice of k is a compromise, and other optimization methods could be evaluated;
- Like Louvain Community Detection, the analysis is restricted to the largest connected component, so isolated days are excluded.

Potential Improvements

- The isolated nodes could be identified as 'isolated' clusters using outlier detection;
- Different features could be weighted differently; some could have a bigger impact on bike demand than others.

Experiment 3: Graph Embeddings

```
import itertools
import random
import warnings
from collections import defaultdict
from typing import Dict, Iterable, List, Tuple

import numpy as np
import pandas as pd
import networkx as nx
from gensim.models import Word2Vec
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler

# Suppress warnings from gensim about lack of C extensions
warnings.filterwarnings("ignore", category=UserWarning,
                        module="gensim")

G_embed = G_largest_cc.copy()

print(G_embed.number_of_nodes(), G_embed.number_of_edges())

710 4618

def compute_spectral_embedding(G: nx.Graph, dim: int) -> np.ndarray:
    """
    Compute the spectral embedding of a graph using the normalized
    Laplacian.

    Parameters
    -----
    G : nx.Graph
        Weighted graph.
    dim : int
```

```

        Target embedding dimension (number of eigenvectors to retain).

Returns
-----
embedding : ndarray of shape (n_nodes, dim)
    Spectral embedding of nodes.
"""
    L = nx.normalized_laplacian_matrix(G,
weight="weight").astype(float)
    eigenvals, eigenvecs = np.linalg.eigh(L.toarray())
    idx = np.argsort(eigenvals)
    eigenvecs = eigenvecs[:, idx]
    embed = eigenvecs[:, 1 : dim + 1]
    return embed

def run_kmeans_embeddings(
    embedding: np.ndarray, cluster_range: Iterable[int], random_state:
int = 42
) -> Tuple[int, Dict[int, np.ndarray], Dict[int, float]]:
    """
    Perform K-means clustering on an embedding for a range of cluster
    numbers.

    Returns the best number of clusters (by silhouette score), a
    mapping of
    cluster assignments for each tested k, and the silhouette scores.
    """
    scores: Dict[int, float] = {}
    assignments: Dict[int, np.ndarray] = {}
    best_k = None
    best_score = -np.inf
    for k in cluster_range:
        if k < 2 or k > embedding.shape[0]:
            continue
        kmeans = KMeans(n_clusters=k, random_state=random_state,
n_init=10)
        labels = kmeans.fit_predict(embedding)
        if len(set(labels)) == 1:
            continue
        score = silhouette_score(embedding, labels)
        scores[k] = score
        assignments[k] = labels
        if score > best_score:
            best_k = k
            best_score = score
    return best_k, assignments, scores

def profile_clusters(
    data_day: pd.DataFrame,
    node_ids: List[int],

```

```

    labels: np.ndarray,
    prefix: str = ""
) -> pd.DataFrame:
    """
    Compute behavioural and contextual summaries for clusters.

    Parameters
    -----
    data_day : pd.DataFrame
        The day-level data (including both contextual and target
features).
    node_ids : list
        List of original node indices corresponding to embedding rows.
    labels : np.ndarray
        Cluster labels for each node.
    prefix : str
        Optional prefix for column names, to differentiate between
methods.

    Returns
    -----
    summary_df : pd.DataFrame
        DataFrame summarising mean values of target and contextual
variables
        per cluster.
    """
    df = data_day.loc[node_ids].copy()
    df[f"{prefix}cluster"] = labels
    behavioural = (
        df.groupby(f"{prefix}cluster")[["cnt", "casual",
"registered"]]
        .mean()
        .rename(columns=lambda x: prefix + x)
    )
    context = (
        df.groupby(f"{prefix}cluster")[
            ["temp", "hum", "windspeed", "weathersit", "season",
"workingday", "holiday", "weekday"]
        ]
        .mean()
        .rename(columns=lambda x: prefix + x)
    )
    summary_df = pd.concat([behavioural, context], axis=1)
    summary_df["size"] = df.groupby(f"{prefix}cluster").size()
    summary_df.sort_values(by="size", ascending=False, inplace=True)
    return summary_df

```

Cell 5: Improved Node2Vec Functions

```
def generate_node2vec_walks(
    G: nx.Graph,
    num_walks: int = 10,
    walk_length: int = 20,
    p: float = 1.0,
    q: float = 1.0,
    seed: int = 42,
) -> List[List[str]]:
    """
    Generate biased random walks for Node2Vec.
    """
    rnd = random.Random(seed)
    neighbors = {u: list(G.neighbors(u)) for u in G.nodes()}

    def node2vec_transition(prev, curr, dst):
        w = G[curr][dst]["weight"]

        if dst == prev:
            return w / p
        elif G.has_edge(dst, prev):
            return w
        else:
            return w / q

    walks = []
    nodes = list(G.nodes())

    for _ in range(num_walks):
        rnd.shuffle(nodes)

        for start in nodes:
            walk = [start]

            while len(walk) < walk_length:
                curr = walk[-1]
                curr_neighbors = neighbors[curr]

                if len(curr_neighbors) == 0:
                    break

                if len(walk) == 1:
                    probs = np.array(
                        [G[curr][nbr]["weight"] for nbr in
curr_neighbors]
                    )
                else:
                    prev = walk[-2]
                    probs = np.array(
```

```

        [node2vec_transition(prev, curr, nbr)
         for nbr in curr_neighbors]
    )

    probs = probs / probs.sum()

    next_node = rnd.choices(
        curr_neighbors,
        weights=probs,
        k=1
    )[0]

    walk.append(next_node)

    walks.append([str(node) for node in walk])

return walks

def train_word2vec_embeddings(
    walks,
    dimensions=16,
    window_size=5,
    workers=1,
    seed=42
):
    """
    Train Word2Vec on random walks.
    """
    model = Word2Vec(
        sentences=walks,
        vector_size=dimensions,
        window=window_size,
        min_count=0,
        sg=1,
        workers=workers,
        epochs=10,
        seed=seed
    )

    return model

def compute_node_embeddings(
    G,
    num_walks=10,
    walk_length=20,
    p=1.0,
    q=1.0,
    dimensions=16,
    window_size=5,

```



```

workers=1,
seed=42
):
    """
    Compute Node2Vec embeddings.
    """

    walks = generate_node2vec_walks(
        G,
        num_walks=num_walks,
        walk_length=walk_length,
        p=p,
        q=q,
        seed=seed
    )

    model = train_word2vec_embeddings(
        walks,
        dimensions=dimensions,
        window_size=window_size,
        workers=workers,
        seed=seed
    )

    embeddings = {}

    for node in G.nodes():
        embeddings[node] = model.wv[str(node)]

    return embeddings

# graph nodes in same order
node_list = list(G_largest_cc.nodes())

# generate node embeddings
node_embeddings = compute_node_embeddings(
    G_largest_cc,
    num_walks=10,
    walk_length=20,
    p=1.0,
    q=1.0,
    dimensions=16,
    window_size=5,
    workers=1,
    seed=42
)

# convert dict to matrix
emb_matrix = np.vstack(

```

```

    [node_embeddings[node] for node in node_list]
)

print("Embedding matrix shape:", emb_matrix.shape)
Embedding matrix shape: (710, 16)

# scale embeddings before DBSCAN
emb_scaled = StandardScaler().fit_transform(emb_matrix)

# initial DBSCAN parameters
dbscan = DBSCAN(
    eps=2.0,
    min_samples=4,
    metric="euclidean"
)

db_labels = dbscan.fit_predict(emb_scaled)

print("Unique cluster labels:", np.unique(db_labels))
print("\nCluster counts:")
print(pd.Series(db_labels).value_counts().sort_index())

# silhouette score only for non-noise points and at least 2 clusters
valid_mask = db_labels != -1
valid_clusters = set(db_labels[valid_mask])

if len(valid_clusters) > 1:
    db_silhouette = silhouette_score(
        emb_scaled[valid_mask],
        db_labels[valid_mask]
    )
    print("\nDBSCAN silhouette score:", round(db_silhouette, 4))
else:
    print("\nSilhouette score not computed (less than 2 non-noise clusters).")

```

Unique cluster labels: [-1 0 1 2 3 4 5 6]

Cluster counts:

-1	32
0	93
1	101
2	297
3	167
4	4
5	12
6	4

dtype: int64

DBSCAN silhouette score: 0.096

```

dbscan_df = data_day.loc[node_list].copy()
dbscan_df["db_cluster"] = db_labels

print("Cluster counts:")
print(dbscan_df["db_cluster"].value_counts().sort_index())
display(dbscan_df.head())

```

Cluster counts:

```

-1      32
0       93
1      101
2      297
3      167
4         4
5       12
6         4

```

Name: db_cluster, dtype: int64

	instant	dteday	season	yr	mnth	holiday	weekday	workingday
0	1	2011-01-01	1	0	1	0	6	0
1	2	2011-01-02	1	0	1	0	0	0
2	3	2011-01-03	1	0	1	0	1	1
3	4	2011-01-04	1	0	1	0	2	1
4	5	2011-01-05	1	0	1	0	3	1

	weathersit	temp	atemp	hum	windspeed	casual
0	2	0.344167	0.363625	0.805833	0.160446	331
1	2	0.363478	0.353739	0.696087	0.248539	131
2	1	0.196364	0.189405	0.437273	0.248309	120
3	1	0.200000	0.212122	0.590435	0.160296	108
4	1	0.226957	0.229270	0.436957	0.186900	82

	cnt	casual_log	registered_log	cluster	db_cluster
0	985	5.805135	6.484635	0	0
1	801	4.882802	6.508769	0	1
2	1349	4.795791	7.114769	1	2
3	1562	4.691348	7.282761	1	2
4	1600	4.418841	7.325808	1	-1

```
dbscan_core = dbscan_df[dbscan_df["db_cluster"] != -1].copy()
```

```
print("Rows excluding noise:", len(dbscan_core))
```

```
print("Clusters excluding noise:",  
sorted(dbscan_core["db_cluster"].unique()))
```

```
Rows excluding noise: 678
```

```
Clusters excluding noise: [0, 1, 2, 3, 4, 5, 6]
```

```
import matplotlib.pyplot as plt
```

```
context_features = [
```

```
    "temp",  
    "hum",  
    "windspeed",  
    "weathersit",  
    "season",  
    "workingday",  
    "holiday",  
    "weekday"
```

```
]
```

```
for col in context_features:
```

```
    plt.figure(figsize=(7, 4))
```

```
    dbscan_core.boxplot(column=col, by="db_cluster")
```

```
    plt.title(f"H1: {col} by DBSCAN cluster")
```

```
    plt.suptitle("")
```

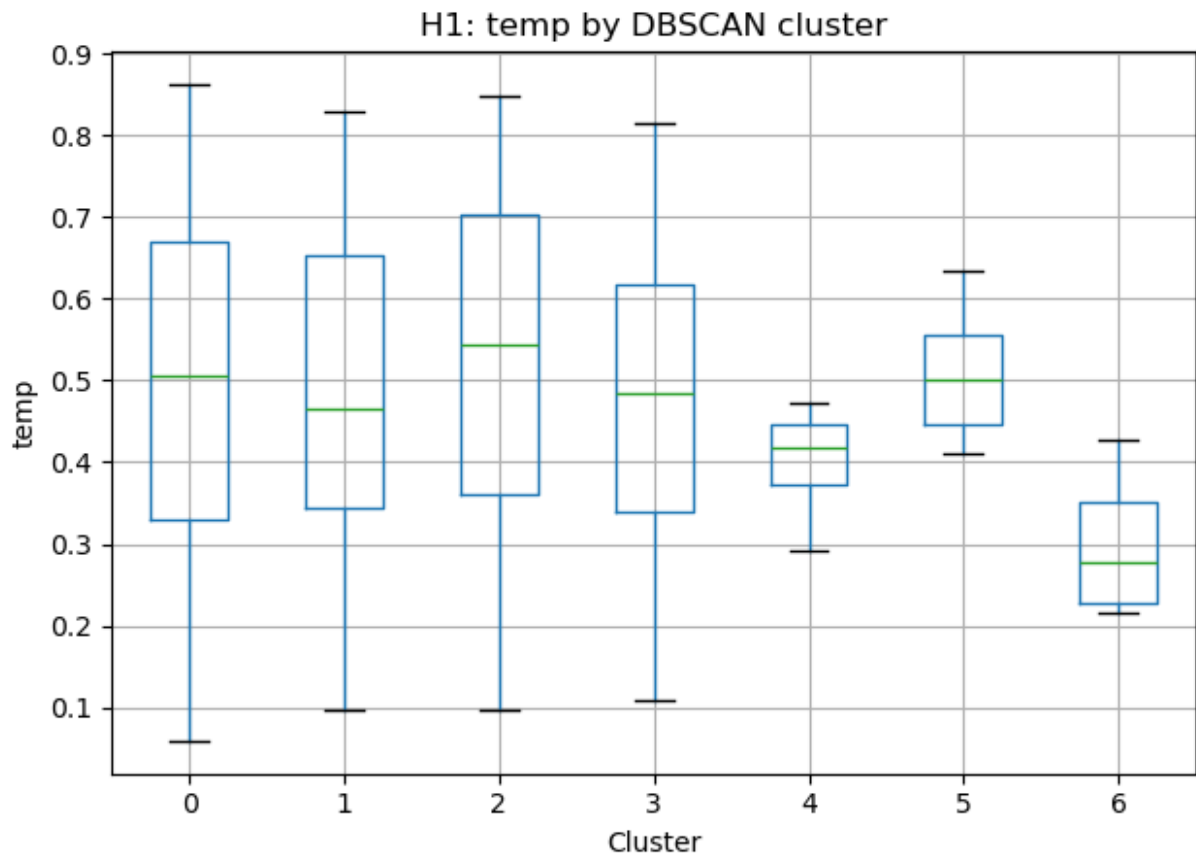
```
    plt.xlabel("Cluster")
```

```
    plt.ylabel(col)
```

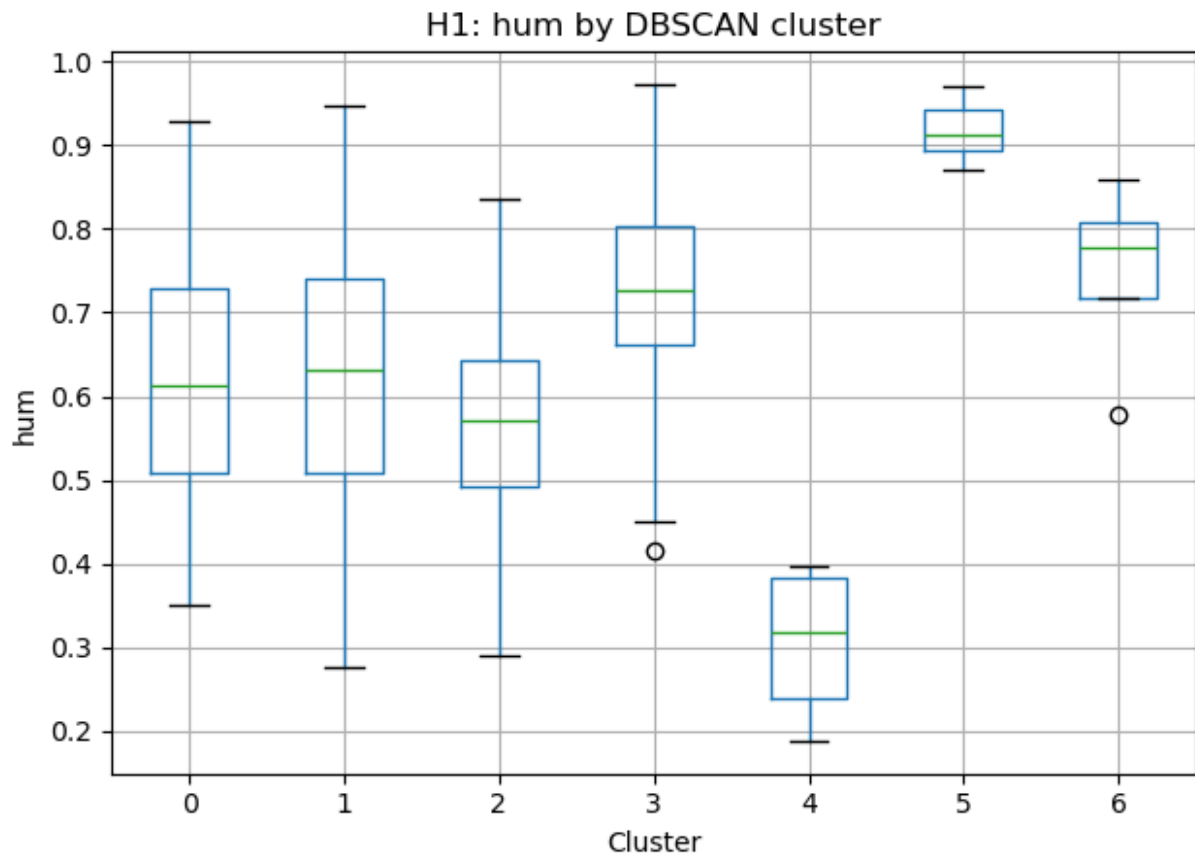
```
    plt.tight_layout()
```

```
    plt.show()
```

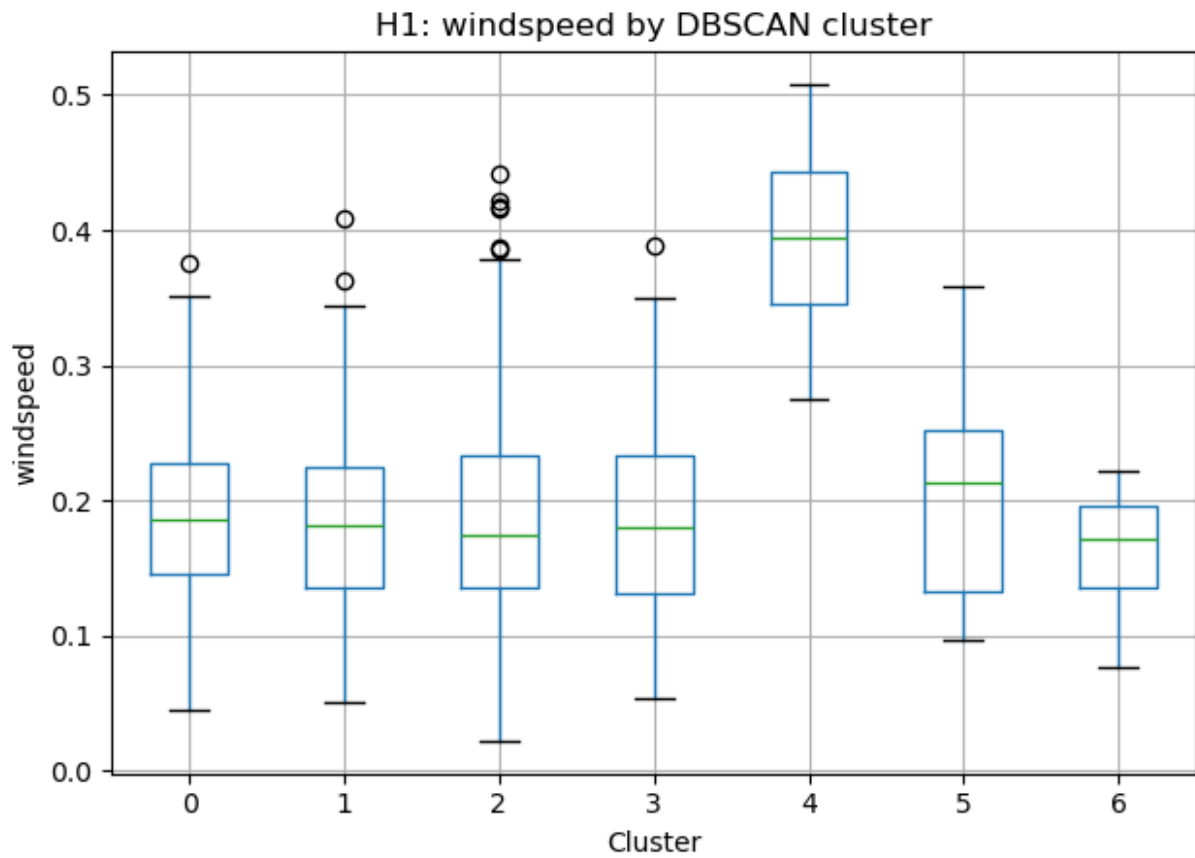
```
<Figure size 700x400 with 0 Axes>
```



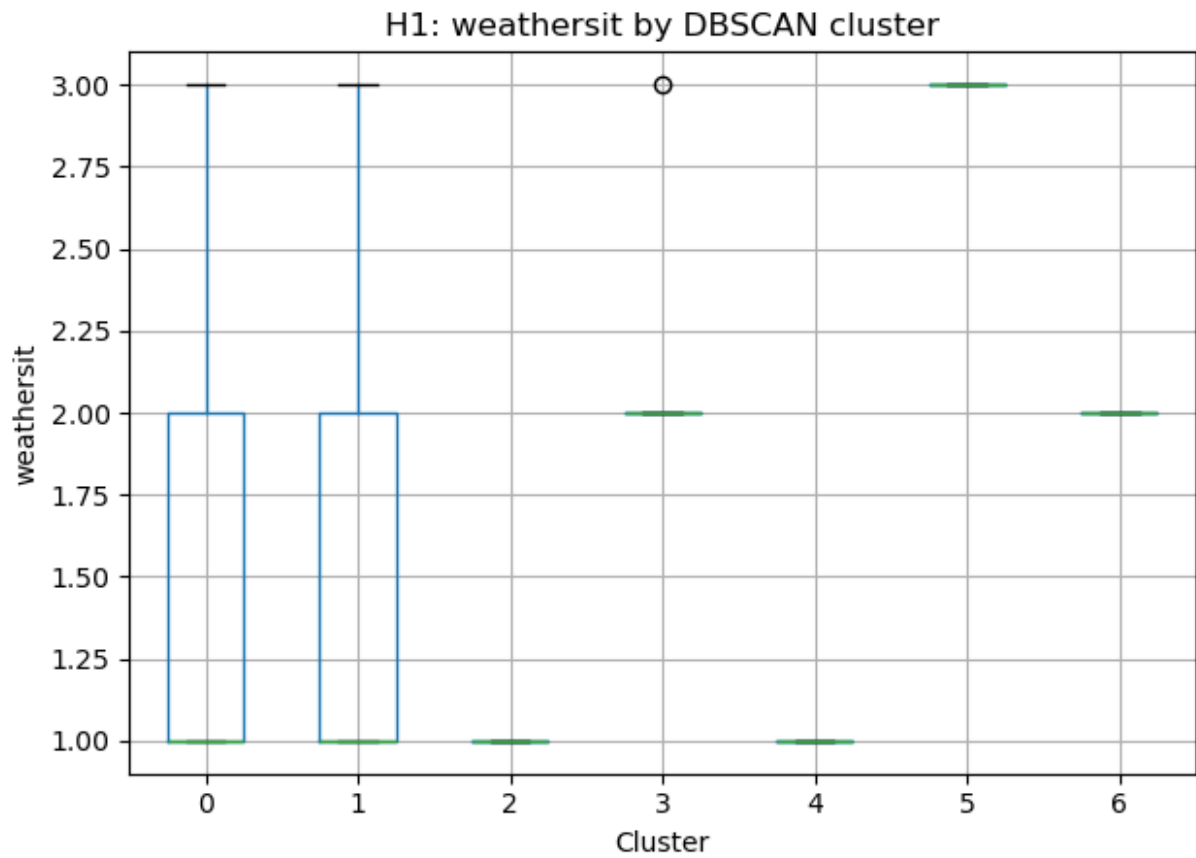
<Figure size 700x400 with 0 Axes>



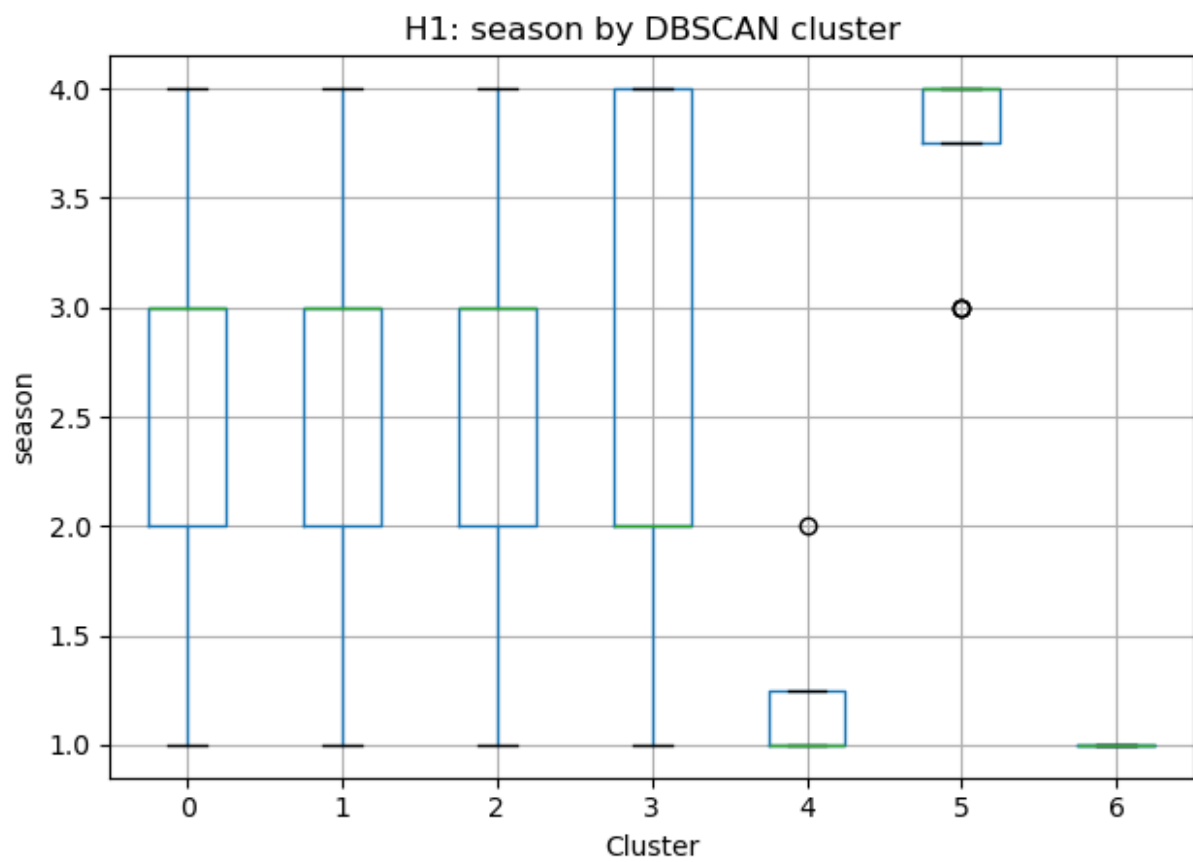
<Figure size 700x400 with 0 Axes>



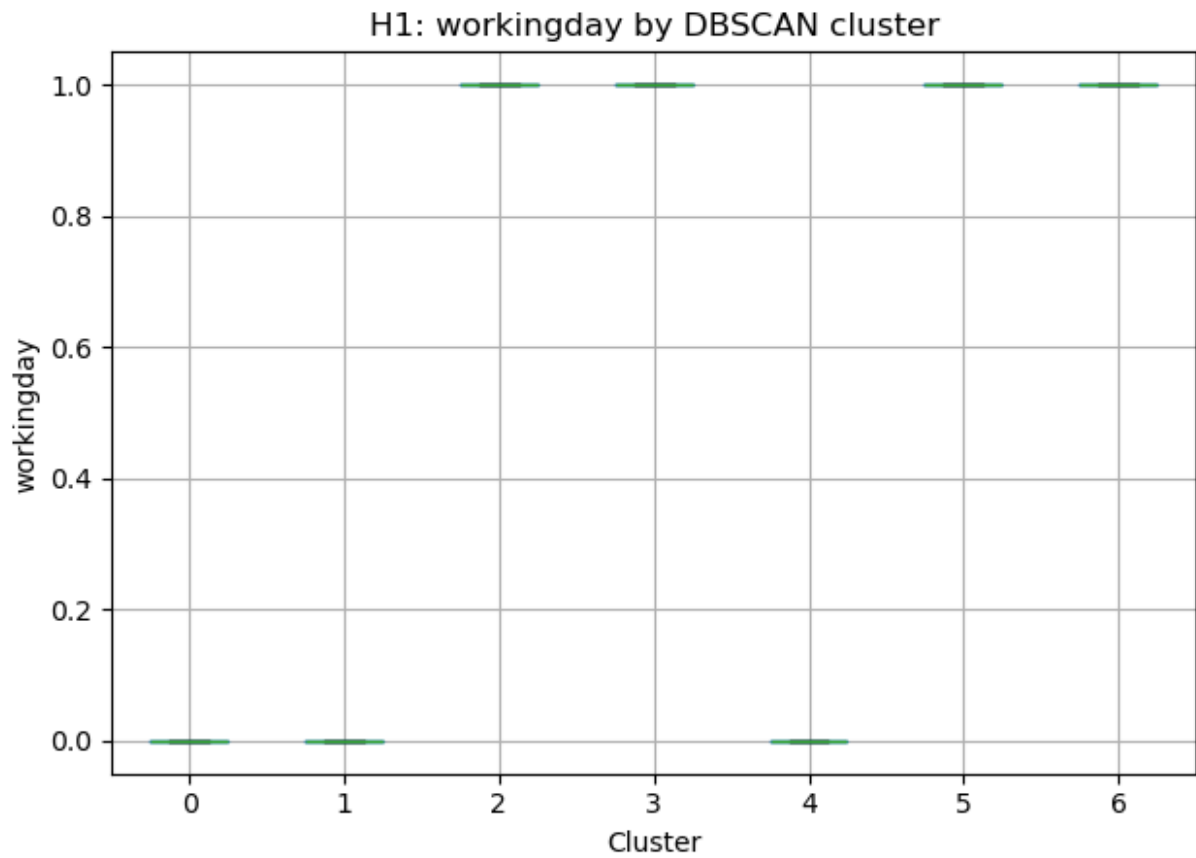
<Figure size 700x400 with 0 Axes>



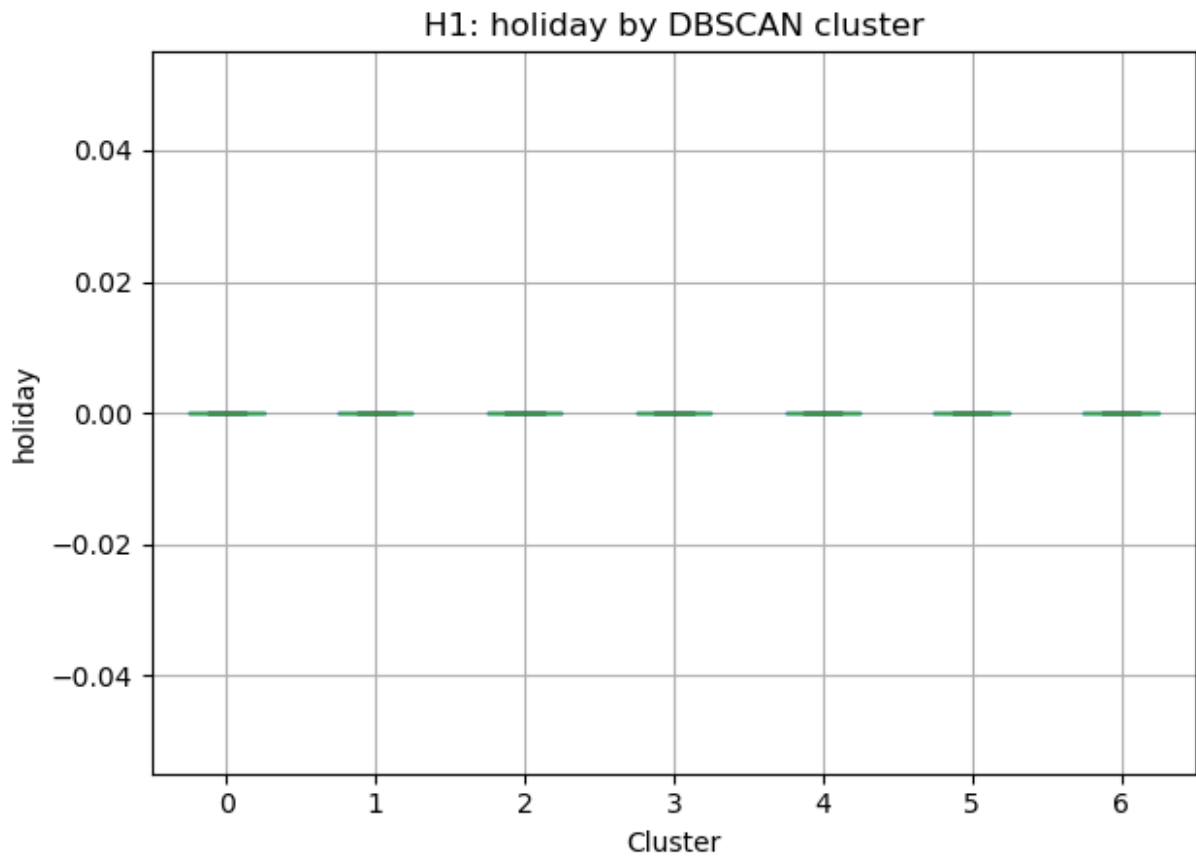
<Figure size 700x400 with 0 Axes>



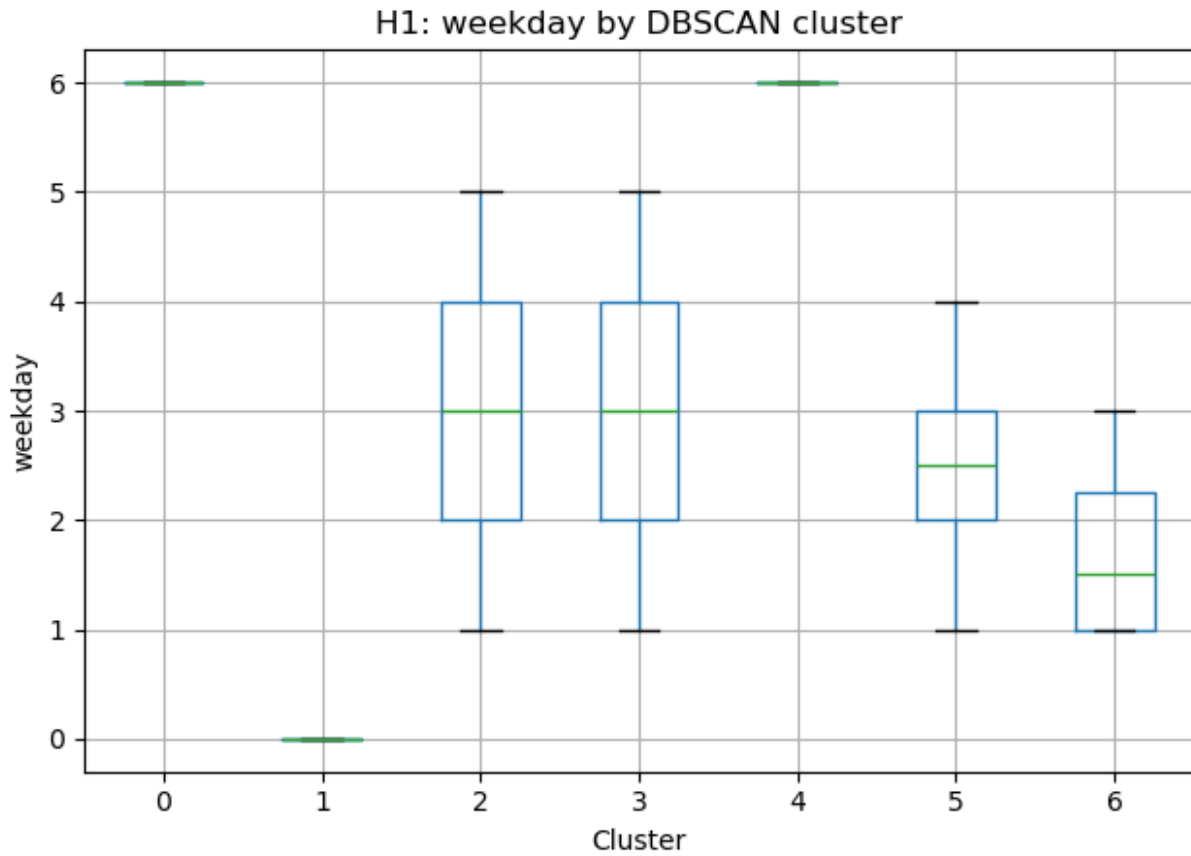
<Figure size 700x400 with 0 Axes>



<Figure size 700x400 with 0 Axes>



<Figure size 700x400 with 0 Axes>



```
from scipy.stats import f_oneway

print("H1 ANOVA results")
for col in [
    "temp", "hum", "windspeed", "weathersit",
    "season", "workingday", "holiday", "weekday"
]:
    groups = [
        grp[col].values
        for _, grp in dbscan_core.groupby("db_cluster")
        if len(grp) > 1
    ]
    if len(groups) >= 2:
        stat, p = f_oneway(*groups)
        print(f"{col:12s} -> F={stat:.4f}, p={p:.6f}")
```

```
H1 ANOVA results
temp      -> F=2.6176, p=0.016251
hum       -> F=48.6457, p=0.000000
windspeed -> F=5.4044, p=0.000018
weathersit -> F=289.9427, p=0.000000
season    -> F=4.8779, p=0.000068
```

```
workingday    -> F=inf, p=0.000000
holiday       -> F=nan, p=nan
weekday       -> F=218.7034, p=0.000000
```

```
/opt/anaconda3/lib/python3.11/site-packages/scipy/stats/_stats_py.py:3895: ConstantInputWarning: Each of the input arrays is constant;the F statistic is not defined or infinite
  warnings.warn(stats.ConstantInputWarning(msg))
/opt/anaconda3/lib/python3.11/site-packages/scipy/stats/_stats_py.py:3895: ConstantInputWarning: Each of the input arrays is constant;the F statistic is not defined or infinite
  warnings.warn(stats.ConstantInputWarning(msg))
```

```
h2_summary = dbscan_core.groupby("db_cluster")["cnt", "casual", "registered"]
              .agg(["mean", "median", "std", "count"]).round(2)

print("H2 Summary: Demand and rider composition by cluster")
display(h2_summary)
```

H2 Summary: Demand and rider composition by cluster

\	cnt			casual			
	mean	median	std	count	mean	median	std
count							
db_cluster							
0	4714.05	4758.0	2181.52	93	1534.74	1504.0	912.31
93							
1	4309.22	4381.0	1843.88	101	1370.77	1415.0	806.30
101							
2	5099.39	4968.0	1741.82	297	711.28	745.0	381.69
297							
3	4130.89	4058.0	1790.24	167	495.56	449.0	366.50
167							
4	3585.25	2924.5	2269.74	4	1381.25	978.0	1336.09
4							
5	2128.08	2206.0	1142.76	12	156.08	149.0	90.45
12							
6	1867.75	1911.0	960.16	4	205.75	140.5	158.87
4							

db_cluster	registered			count
	mean	median	std	
0	3179.31	3195.0	1345.62	93
1	2938.45	2919.0	1156.80	101
2	4388.11	4196.0	1458.62	297
3	3635.33	3571.0	1498.38	167

4	2204.00	2054.0	1077.13	4
5	1972.00	2024.0	1061.48	12
6	1662.00	1674.5	892.74	4

```
h2_ratio = dbscan_core.groupby("db_cluster")[["casual", "registered",
"cnt"]].mean().round(2)
h2_ratio["casual_share"] = (h2_ratio["casual"] /
h2_ratio["cnt"]).round(3)
h2_ratio["registered_share"] = (h2_ratio["registered"] /
h2_ratio["cnt"]).round(3)
```

```
print("H2 Ratio Summary")
display(h2_ratio)
```

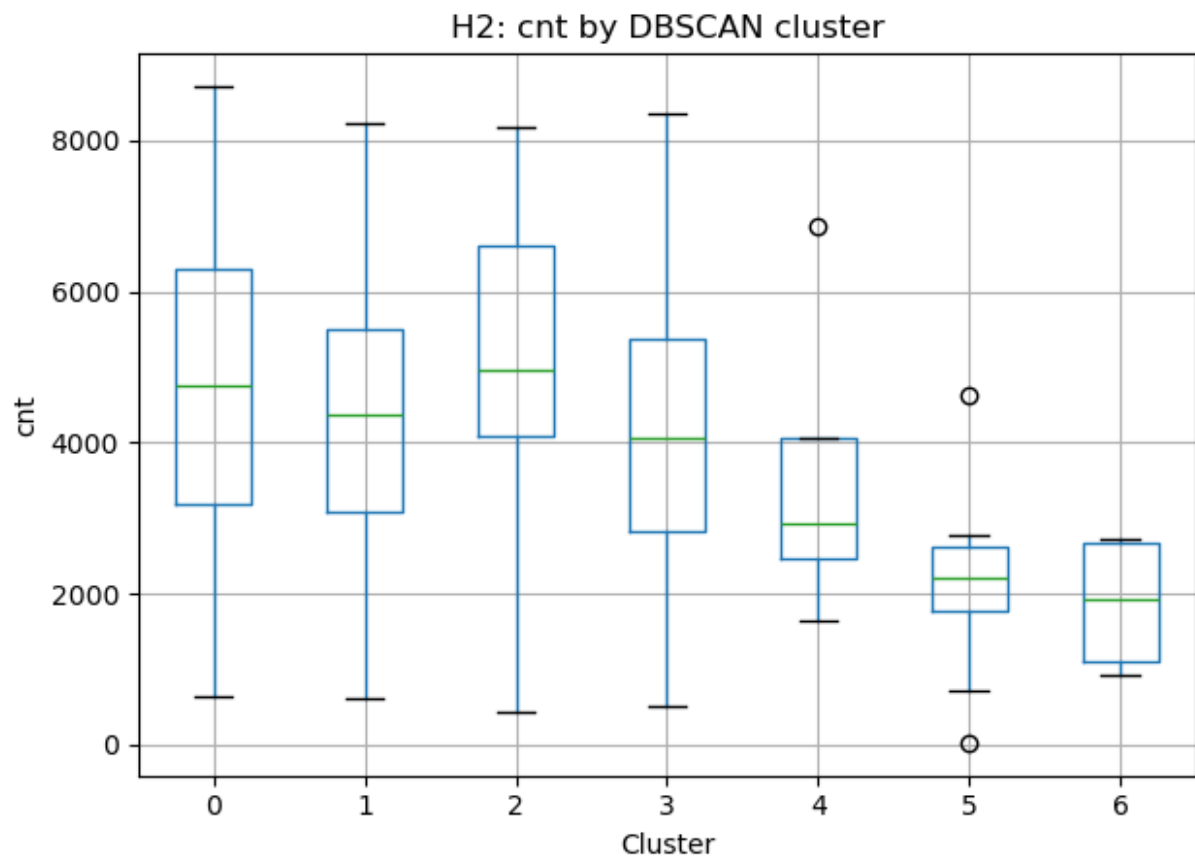
H2 Ratio Summary

	casual	registered	cnt	casual_share
registered_share				
db_cluster				
0	1534.74	3179.31	4714.05	0.326
0.674				
1	1370.77	2938.45	4309.22	0.318
0.682				
2	711.28	4388.11	5099.39	0.139
0.861				
3	495.56	3635.33	4130.89	0.120
0.880				
4	1381.25	2204.00	3585.25	0.385
0.615				
5	156.08	1972.00	2128.08	0.073
0.927				
6	205.75	1662.00	1867.75	0.110
0.890				

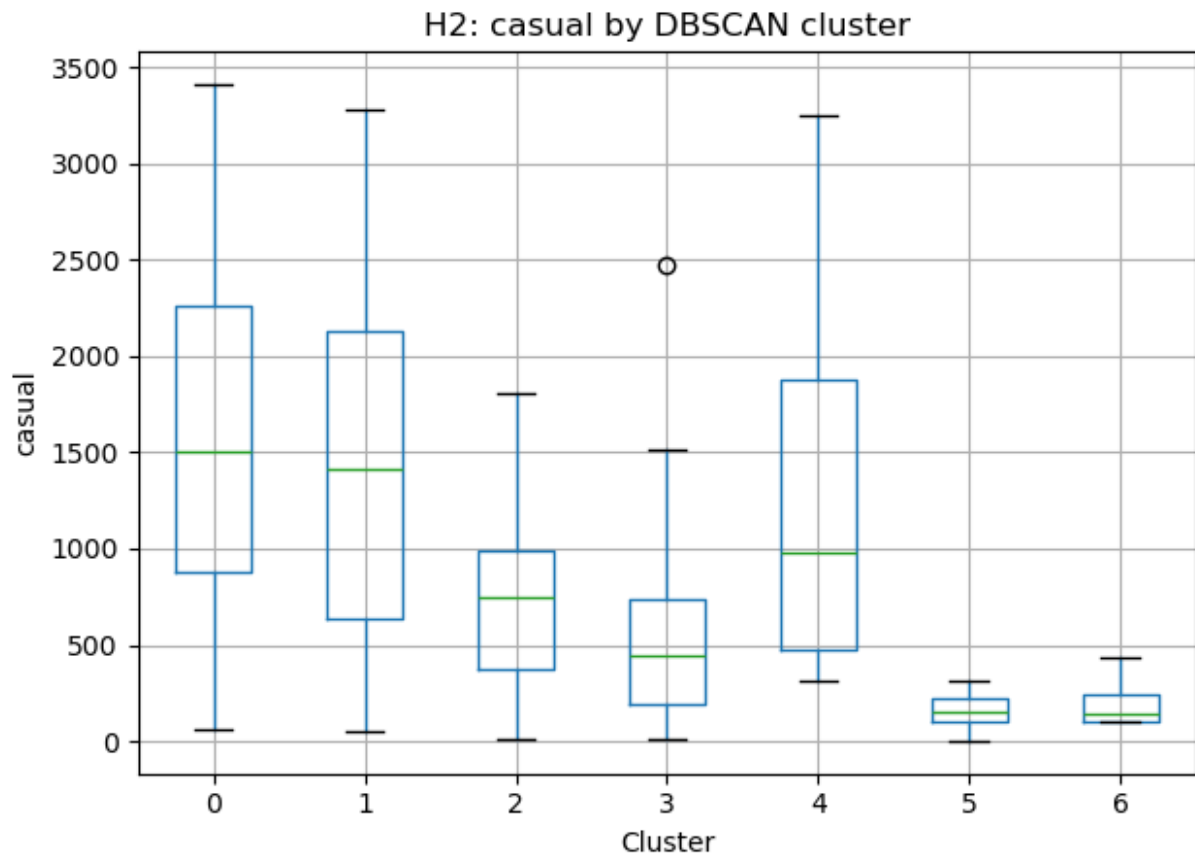
```
behavior_features = ["cnt", "casual", "registered"]
```

```
for col in behavior_features:
    plt.figure(figsize=(7, 4))
    dbscan_core.boxplot(column=col, by="db_cluster")
    plt.title(f"H2: {col} by DBSCAN cluster")
    plt.suptitle("")
    plt.xlabel("Cluster")
    plt.ylabel(col)
    plt.tight_layout()
    plt.show()
```

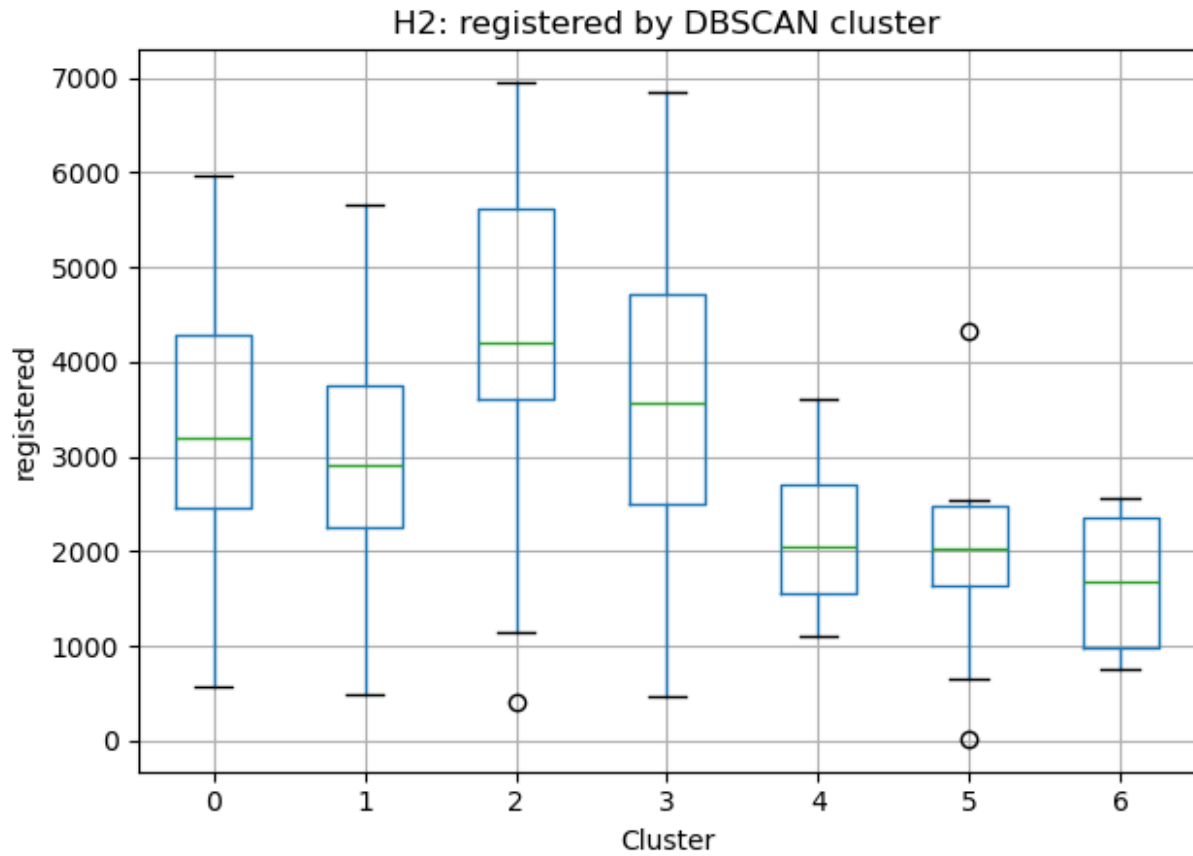
<Figure size 700x400 with 0 Axes>



<Figure size 700x400 with 0 Axes>



<Figure size 700x400 with 0 Axes>



```
print("H2 ANOVA results")
for col in ["cnt", "casual", "registered"]:
    groups = [
        grp[col].values
        for _, grp in dbscan_core.groupby("db_cluster")
        if len(grp) > 1
    ]
    if len(groups) >= 2:
        stat, p = f_oneway(*groups)
        print(f"{col:12s} -> F={stat:.4f}, p={p:.6f}")
```

H2 ANOVA results

```
cnt          -> F=11.3712, p=0.000000
casual       -> F=55.8757, p=0.000000
registered   -> F=24.1147, p=0.000000
```

```
h3_summary = dbscan_core.groupby("db_cluster")[
    [
        "cnt",
        "casual",
        "registered",
```

```

        "temp",
        "hum",
        "windspeed",
        "weathersit",
        "season",
        "workingday",
        "holiday",
        "weekday"
    ]
].mean().round(3)

print("H3 Summary: Mean profile per cluster")
display(h3_summary)

```

H3 Summary: Mean profile per cluster

db_cluster	cnt	casual	registered	temp	hum	windspeed	\
0	4714.054	1534.742	3179.312	0.497	0.629	0.188	
1	4309.218	1370.772	2938.446	0.490	0.629	0.186	
2	5099.394	711.279	4388.114	0.527	0.567	0.189	
3	4130.886	495.557	3635.329	0.477	0.723	0.186	
4	3585.250	1381.250	2204.000	0.400	0.304	0.393	
5	2128.083	156.083	1972.000	0.507	0.918	0.207	
6	1867.750	205.750	1662.000	0.300	0.747	0.160	

db_cluster	weathersit	season	workingday	holiday	weekday
0	1.409	2.527	0.0	0.0	6.000
1	1.307	2.515	0.0	0.0	0.000
2	1.000	2.539	1.0	0.0	3.044
3	2.006	2.533	1.0	0.0	3.072
4	1.000	1.250	0.0	0.0	6.000
5	3.000	3.750	1.0	0.0	2.417
6	2.000	1.000	1.0	0.0	1.750

```

overall_cnt_mean = dbscan_core["cnt"].mean()

for cluster_id, row in h3_summary.iterrows():
    print(f"\nCluster {cluster_id}")

    if row["workingday"] > 0.7 and row["registered"] > row["casual"]:
        print("Suggested label: commuter-dominant working days")

    elif row["workingday"] < 0.4 and row["casual"] > 0.35 *
row["registered"]:
        print("Suggested label: leisure-dominant weekends/holidays")

    elif row["cnt"] < overall_cnt_mean:
        print("Suggested label: low-demand days")

```

```
else:
    print("Suggested label: mixed/transitional regime")

print("Profile:")
print(row)
```

Cluster 0

Suggested label: leisure-dominant weekends/holidays

Profile:

cnt	4714.054
casual	1534.742
registered	3179.312
temp	0.497
hum	0.629
windspeed	0.188
weathersit	1.409
season	2.527
workingday	0.000
holiday	0.000
weekday	6.000

Name: 0, dtype: float64

Cluster 1

Suggested label: leisure-dominant weekends/holidays

Profile:

cnt	4309.218
casual	1370.772
registered	2938.446
temp	0.490
hum	0.629
windspeed	0.186
weathersit	1.307
season	2.515
workingday	0.000
holiday	0.000
weekday	0.000

Name: 1, dtype: float64

Cluster 2

Suggested label: commuter-dominant working days

Profile:

cnt	5099.394
casual	711.279
registered	4388.114
temp	0.527
hum	0.567
windspeed	0.189
weathersit	1.000

```
season          2.539
workingday      1.000
holiday         0.000
weekday        3.044
Name: 2, dtype: float64
```

Cluster 3

Suggested label: commuter-dominant working days

Profile:

```
cnt          4130.886
casual       495.557
registered   3635.329
temp         0.477
hum          0.723
windspeed    0.186
weathersit    2.006
season       2.533
workingday   1.000
holiday      0.000
weekday      3.072
Name: 3, dtype: float64
```

Cluster 4

Suggested label: leisure-dominant weekends/holidays

Profile:

```
cnt          3585.250
casual       1381.250
registered   2204.000
temp         0.400
hum          0.304
windspeed    0.393
weathersit    1.000
season       1.250
workingday   0.000
holiday      0.000
weekday      6.000
Name: 4, dtype: float64
```

Cluster 5

Suggested label: commuter-dominant working days

Profile:

```
cnt          2128.083
casual       156.083
registered   1972.000
temp         0.507
hum          0.918
windspeed    0.207
weathersit    3.000
season       3.750
workingday   1.000
```

```
holiday      0.000
weekday      2.417
Name: 5, dtype: float64
```

Cluster 6

Suggested label: commuter-dominant working days

Profile:

```
cnt          1867.750
casual        205.750
registered    1662.000
temp          0.300
hum           0.747
windspeed     0.160
weathersit     2.000
season         1.000
workingday     1.000
holiday        0.000
weekday        1.750
Name: 6, dtype: float64
```

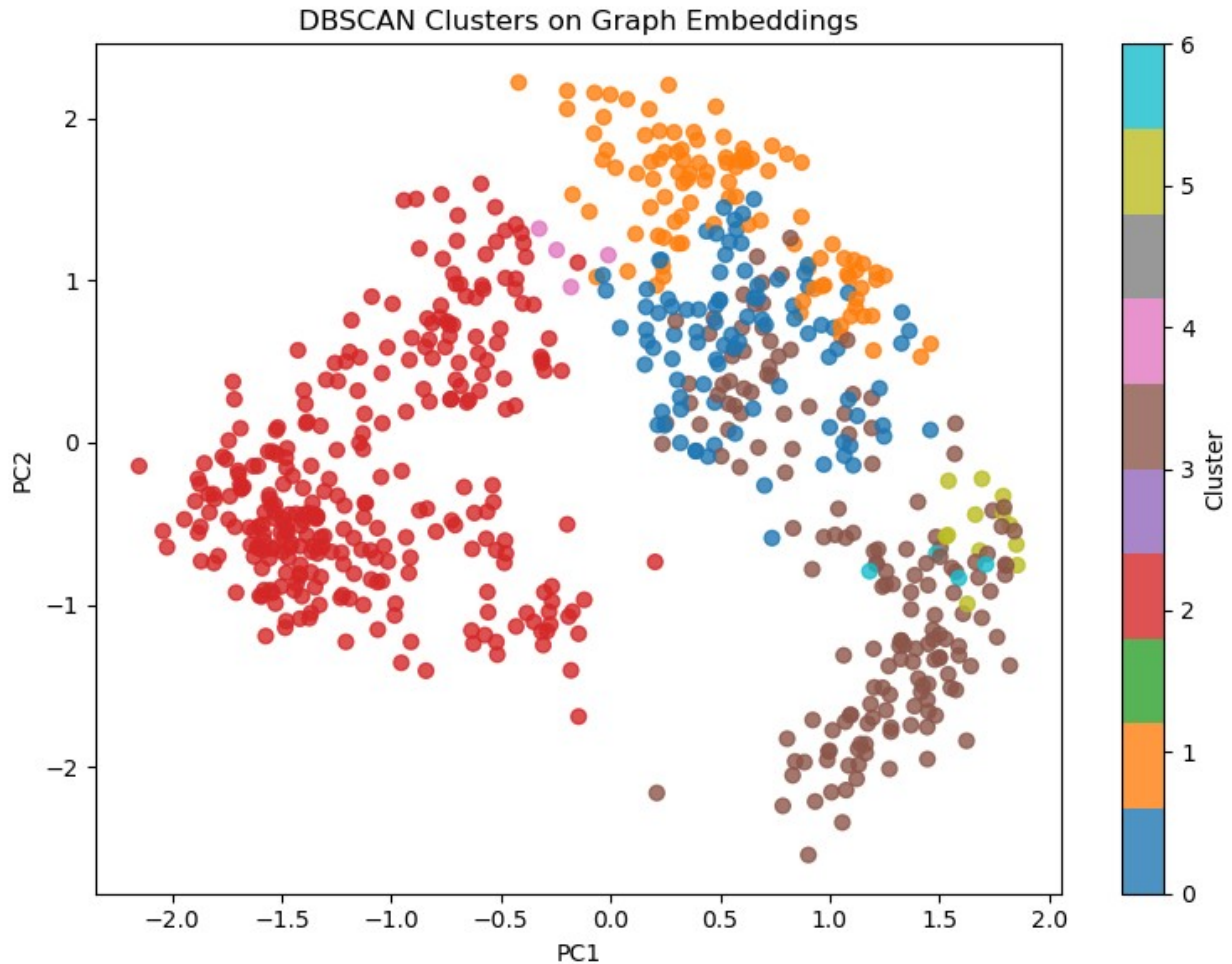
```
from sklearn.decomposition import PCA
```

```
emb_core = emb_matrix[db_labels != -1]
labels_core = db_labels[db_labels != -1]
```

```
pca = PCA(n_components=2, random_state=42)
emb_2d = pca.fit_transform(emb_core)
```

```
plt.figure(figsize=(8, 6))
scatter = plt.scatter(
    emb_2d[:, 0],
    emb_2d[:, 1],
    c=labels_core,
    cmap="tab10",
    s=40,
    alpha=0.8
)
```

```
plt.title("DBSCAN Clusters on Graph Embeddings")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.colorbar(scatter, label="Cluster")
plt.tight_layout()
plt.show()
```



#Experiment 3 -Discussion

Hypothesis H1 was strongly supported. One-way ANOVA showed statistically significant differences across clusters for temperature ($p = 0.016$), humidity ($p < 0.001$), windspeed ($p < 0.001$), weather situation ($p < 0.001$), season ($p < 0.001$), workingday status ($p < 0.001$), and weekday ($p < 0.001$). This indicates that the graph-based communities capture coherent weather and calendar conditions rather than arbitrary partitions.

Hypothesis H2 was strongly supported. One-way ANOVA indicated significant differences between clusters in total rentals (cnt, $F = 11.37$, $p < 0.001$), casual riders (casual, $F = 55.88$, $p < 0.001$), and registered riders (registered, $F = 24.11$, $p < 0.001$). The strongest separation was observed for casual demand, suggesting that clusters effectively capture differences between leisure-oriented and commuter-oriented day types.

Hypothesis H3 was strongly supported. The detected communities corresponded to practical and interpretable demand regimes rather than arbitrary partitions. Distinct weekend clusters captured leisure-oriented Saturday and Sunday usage with elevated casual demand, while weekday clusters represented commuter-dominant conditions with high registered ridership. Additional clusters reflected suppressed-demand scenarios such as cold days and high-humidity / poor-weather days. These results demonstrate that the graph-based communities recover meaningful recurring day types relevant to bike-sharing demand behavior.

Limitations

The experiment, however, had some problems. The quality of the embeddings was highly contingent on the construction of the original similarity graph, encompassing the chosen features, distance metric, and nearest-neighbor configurations. Also, both Node2Vec and DBSCAN needed careful tuning of their hyperparameters because earlier runs either labelled all points as noise or made too many broken clusters. The analysis also looked at each day on its own and didn't try to model things like weekly cycles or long-term trends that depend on time. Lastly, the results only apply to the Washington D.C. bike-sharing dataset and may not apply to other cities or transportation systems. In the future, we could test different graph embedding methods, dynamic temporal graphs, external contextual variables, and supervised forecasting tasks that use the learned embeddings.

Conclusion

The three graph-based experiments all come to the same main conclusion: the day-level similarity graph has meaningful demand regimes, not random partitions. Louvain Community Detection, Spectral Clustering, and the graph embeddings experiment (Node2Vec embeddings with DBSCAN clustering) all work on different ideas: local modular structure, global spectral structure, and latent representation learning with density-based clustering. However, all three methods found groups that separate days by demand intensity and rider composition. The graph embeddings experiment specifically uncovered practical regimes, including commuter-dominant weekdays, leisure-focused weekends, and weather-influenced low-demand days. This agreement indicates that the observed graph structure is an intrinsic characteristic of the dataset rather than a byproduct of any specific clustering technique.

The results demonstrate that graph mining is effective for analysing bike demand, as it associates structural patterns (i.e., the connections between days in the similarity graph) with behavioural interpretation (i.e., the variations in cnt, casual, and registered communities). In all three experiments, the detected regimes consistently differentiated between commuter-oriented and leisure-oriented usage across varying weather and calendar conditions. The embeddings-based method also showed that these patterns are still clear in a low-dimensional latent space, where density-based clustering made a smaller and easier-to-understand set of recurring day types. Nonetheless, the results are contingent upon graph-construction parameters (feature set, k , similarity weighting), embedding and clustering hyperparameters, and the choice to limit the analysis to the largest connected component.

How feedback from Checkpoint 1 was addressed:

- The task and research questions have been clearly defined by narrowing the objective and setting up some carefully chosen hypotheses in relation to the dataset.
- Checkpoint 2 moved from relying on simply presenting the result to a more insight-driven approach by explicitly linking the findings to hypotheses H1-H3.
- Modelling choices is explained with the same rigor as the previous hyperparameter discussion, as this was highlighted as a good.
- The individual results summarized in a conclusion section to textually explaining findings.
- The experiments conducted significantly expanded.

4. Module 3 — Pattern / Text Mining

```
# Pattern mining implementation
```

```
patterns = None
```

5. Final Synthesis and Reflection

- Key insights:
- Limitations:
- Revisions after feedback: